BUILD-IT
A MODULAR ROBOTICS KIT

BY: IZN SIDDIQUI

Problem Statement

- Lack of Affordable and Locally Available STEM Kits in Pakistan

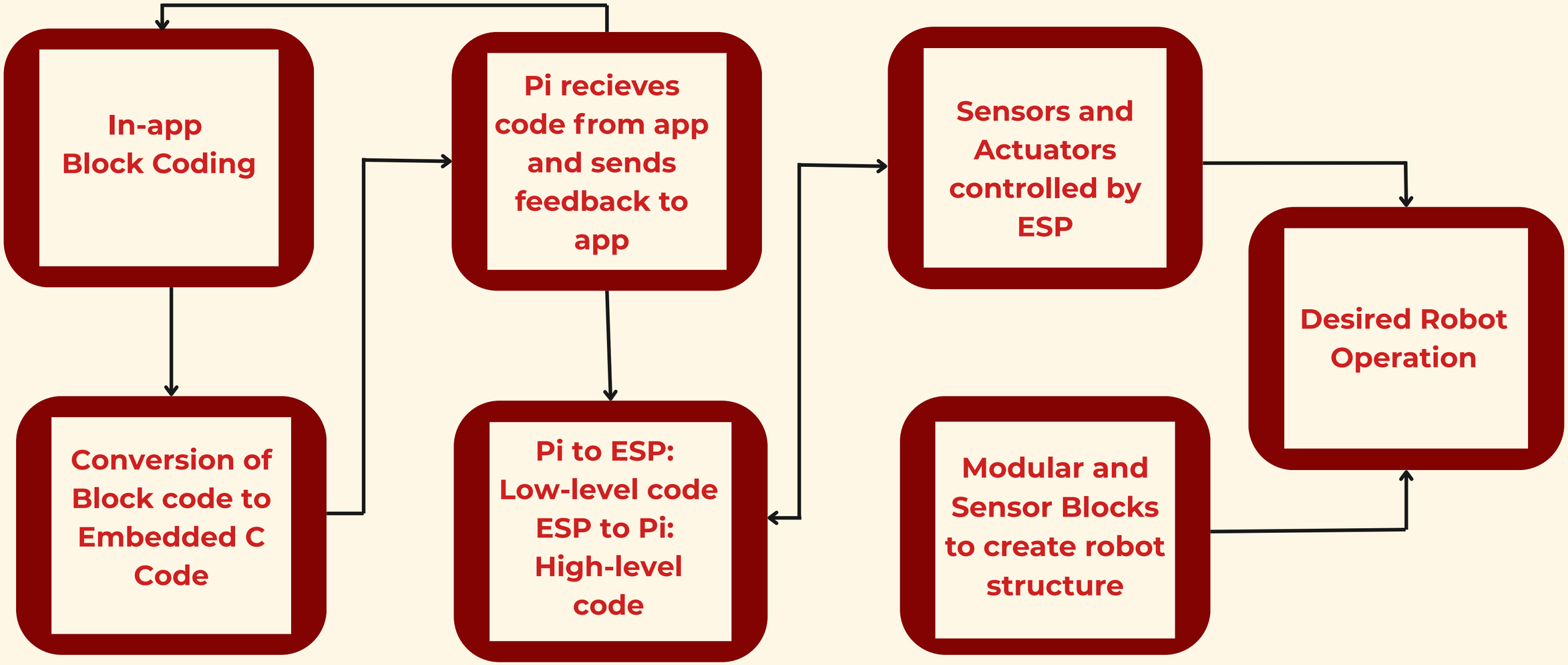
- Limited Access to Practical Learning in the realm of Machine Learning





Goal:

To create a user-friendly modular robotics kit with easy-to-use designs, multiple sensors integration and Machine Learning education for STEM students in Pakistan.


In-app
Block Coding
Pi recieves code from app and sends feedback to app
Sensors and Actuators controlled by ESP
Desired Robot Operation
Conversion of Block code to Embedded C Code
Pi to ESP: Low-level code
ESP to Pi: High-level code
Modular and Sensor Blocks to create robot structure

HARDWARE ELEMENTS

Hardware Elements

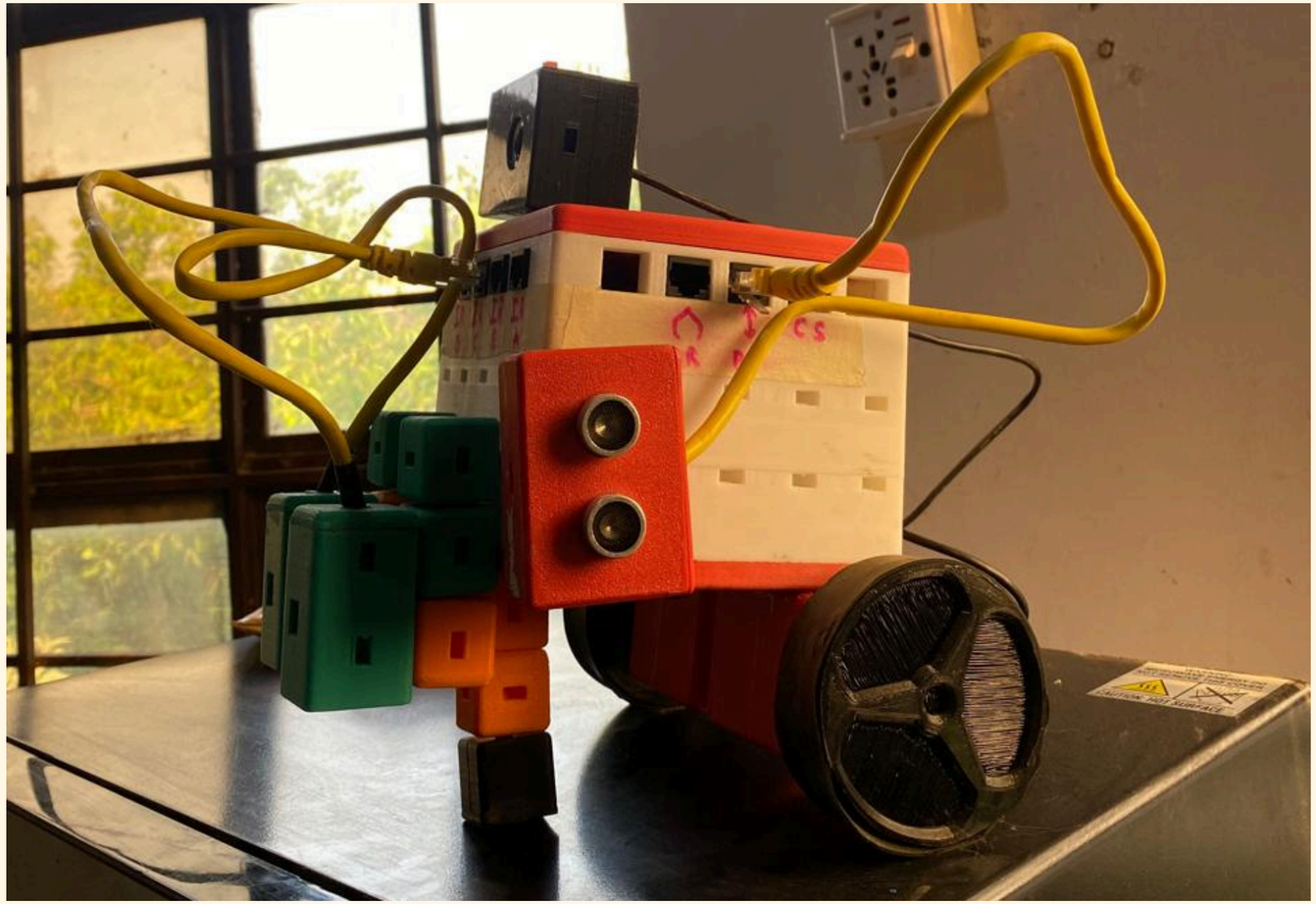
01 Power Brick

02 Modular Building Blocks

03 Snap-fit Pin

04 2 DOF Gripper

05 Sensor and Motor Blocks

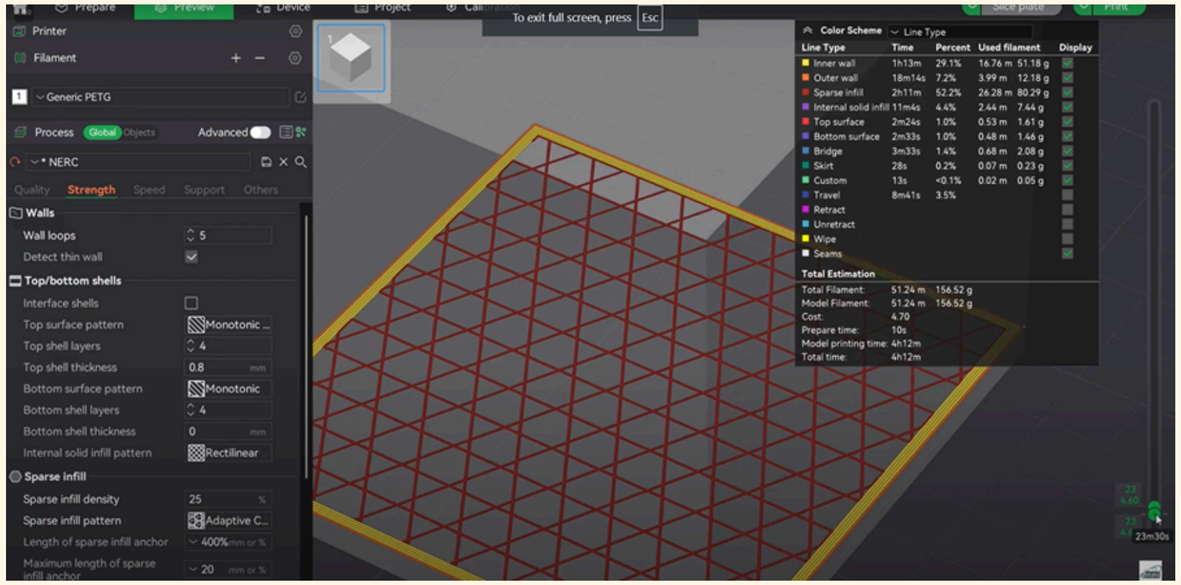


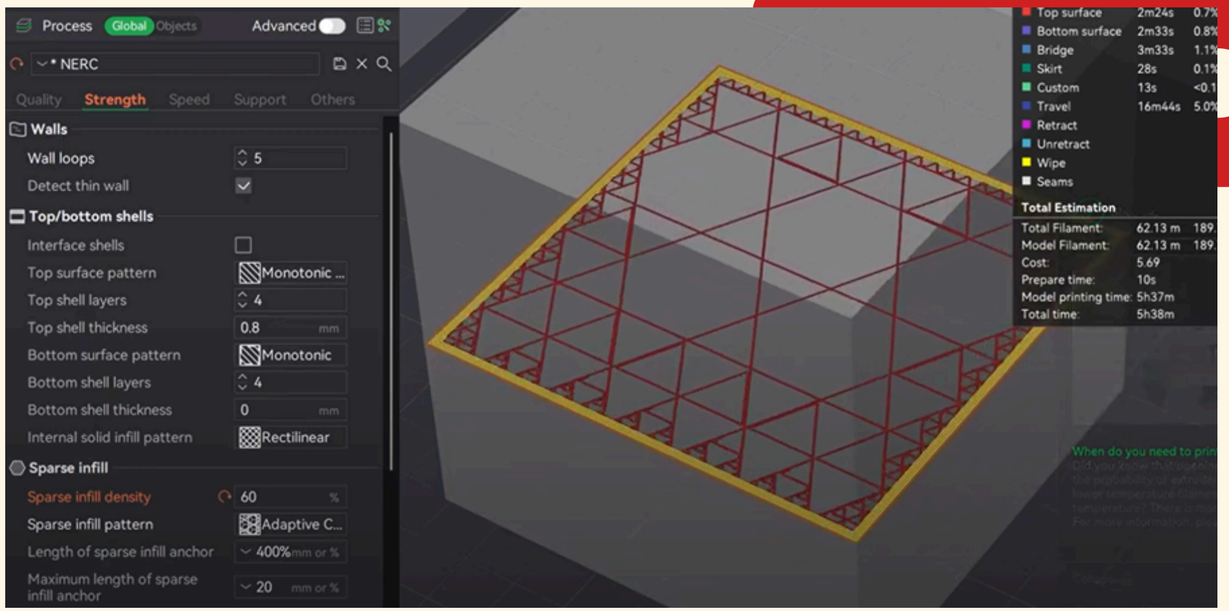
Material Selection

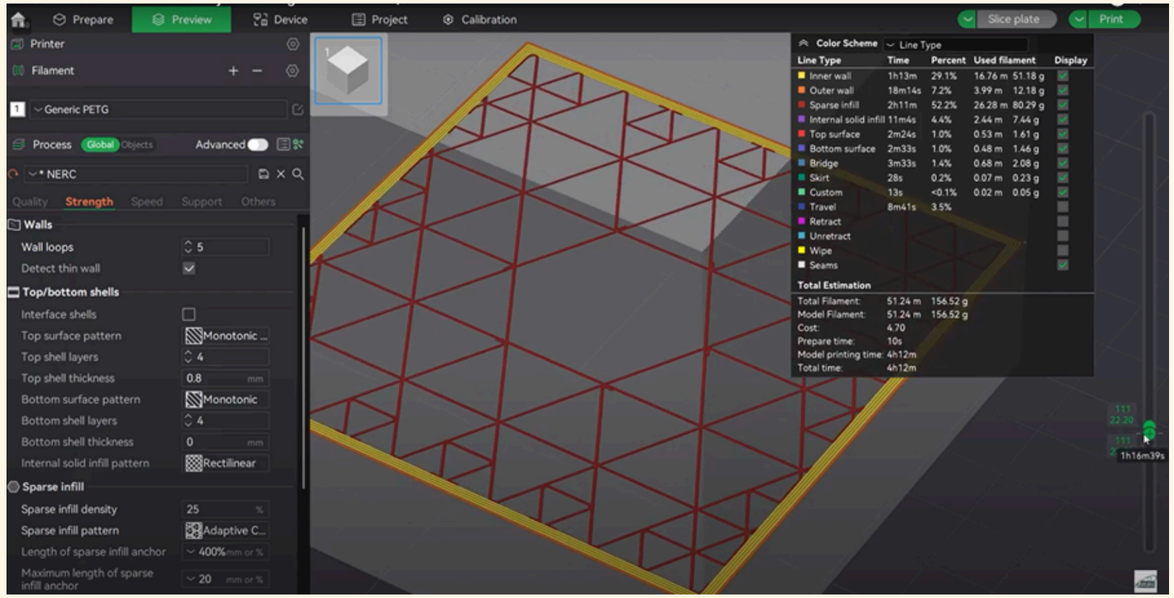
- PLA over PETG as PLA has high stiffness and print accuracy.
- 20% Cheaper
- Lower warping so better print accuracy
- Better finish

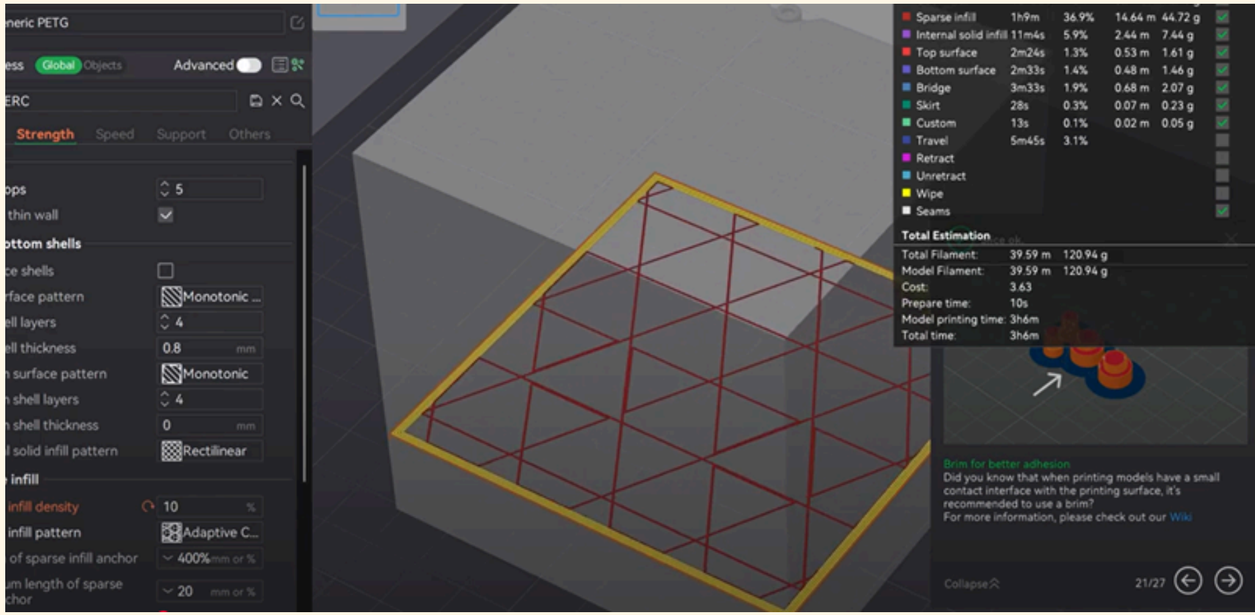
Printing Specifications

- 25% infill , except the connecting rod
- Adaptive Cubic Pattern, more control over weight distribution


To exit full screen, press Esc
Prepare
Preview
Device
Project
Calibration
Slice plate
Print
Printer
Filament
Generic PETG
Process
Global
Objects
Advanced
* NERC
Quality
Strength
Speed
Support
Others
Walls
Wall loops 5
Detect thin wall
Top/bottom shells
Interface shells
Top surface pattern Monotonic ...
Top shell layers 4
Top shell thickness 0.8 mm
Bottom surface pattern Monotonic
Bottom shell layers 4
Bottom shell thickness 0 mm
Internal solid infill pattern Rectilinear
Sparse infill
Sparse infill density 25 %
Sparse infill pattern Adaptive C...
Length of sparse infill anchor 400% mm or %
Maximum length of sparse infill anchor 20 mm or %
Color Scheme Line Type
Line Type Time Percent Used filament Display
Inner wall 1h13m 29.1% 16.76 m 51.18 g
Outer wall 18m14s 7.2% 3.99 m 12.18 g
Sparse infill 2h11m 52.2% 26.28 m 80.29 g
Internal solid infill 11m4s 4.4% 2.44 m 7.44 g
Top surface 2m24s 1.0% 0.53 m 1.61 g
Bottom surface 2m33s 1.0% 0.48 m 1.46 g
Bridge 3m33s 1.4% 0.68 m 2.08 g
Skirt 28s 0.2% 0.07 m 0.23 g
Custom 13s <0.1% 0.02 m 0.05 g
Travel 8m41s 3.5%
Retract
Unretract
Wipe
Seams
Total Estimation
Total Filament: 51.24 m 156.52 g
Model Filament: 51.24 m 156.52 g
Cost: 4.70
Prepare time: 10s
Model printing time: 4h12m
Total time: 4h12m
23m30s

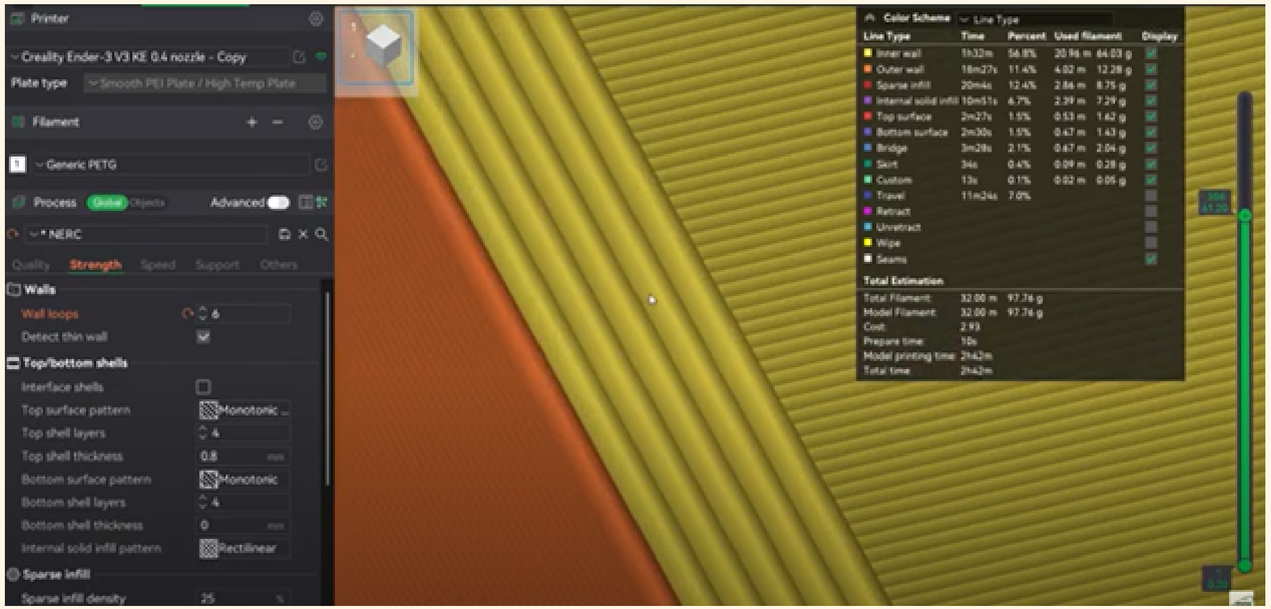

Process
Global
Objects
Advanced
* NERC
Quality
Strength
Speed
Support
Others
Walls
Wall loops 5
Detect thin wall
Top/bottom shells
Interface shells
Top surface pattern Monotonic ...
Top shell layers 4
Top shell thickness 0.8 mm
Bottom surface pattern Monotonic
Bottom shell layers 4
Bottom shell thickness 0 mm
Internal solid infill pattern Rectilinear
Sparse infill
Sparse infill density 60 %
Sparse infill pattern Adaptive C...
Length of sparse infill anchor 400% mm or %
Maximum length of sparse infill anchor 20 mm or %
Top surface 2m24s 0.7%
Bottom surface 2m33s 0.8%
Bridge 3m33s 1.1%
Skirt 28s 0.1%
Custom 13s <0.1
Travel 16m44s 5.0%
Retract
Unretract
Wipe
Seams
Total Estimation
Total Filament: 62.13 m 189
Model Filament: 62.13 m 189
Cost: 5.69
Prepare time: 10s
Model printing time: 5h37m
Total time: 5h38m

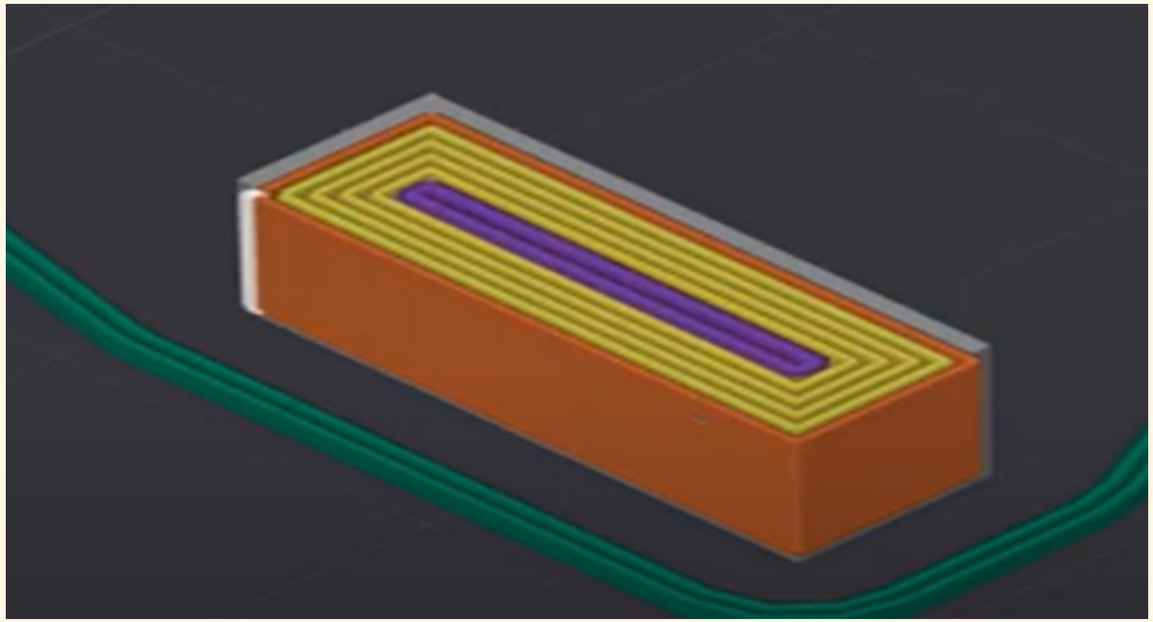

Prepare
Preview
Device
Project
Calibration
Slice plate
Print
Printer
Filament
Generic PETG
Process
Global
Objects
Advanced
* NERC
Quality
Strength
Speed
Support
Others
Walls
Wall loops 5
Detect thin wall
Top/bottom shells
Interface shells
Top surface pattern Monotonic ...
Top shell layers 4
Top shell thickness 0.8 mm
Bottom surface pattern Monotonic
Bottom shell layers 4
Bottom shell thickness 0 mm
Internal solid infill pattern Rectilinear
Sparse infill
Sparse infill density 25 %
Sparse infill pattern Adaptive C...
Length of sparse infill anchor 400% mm or %
Maximum length of sparse infill anchor 20 mm or %
Color Scheme Line Type
Line Type Time Percent Used filament Display
Inner wall 1h13m 29.1% 16.76 m 51.18 g
Outer wall 18m14s 7.2% 3.99 m 12.18 g
Sparse infill 2h11m 52.2% 26.28 m 80.29 g
Internal solid infill 11m4s 4.4% 2.44 m 7.44 g
Top surface 2m24s 1.0% 0.53 m 1.61 g
Bottom surface 2m33s 1.0% 0.48 m 1.46 g
Bridge 3m33s 1.4% 0.68 m 2.08 g
Skirt 28s 0.2% 0.07 m 0.23 g
Custom 13s <0.1% 0.02 m 0.05 g
Travel 8m41s 3.5%
Retract
Unretract
Wipe
Seams
Total Estimation
Total Filament: 51.24 m 156.52 g
Model Filament: 51.24 m 156.52 g
Cost: 4.70
Prepare time: 10s
Model printing time: 4h12m
Total time: 4h12m
1h16m39s


Strength
Speed
Support
Others
5
0.8 mm
Monotonic
4
0 mm
Rectilinear
10 %
Adaptive C...
400% mm or %
20 mm or %
Sparse infill 1h9m 36.9% 14.64 m 44.72 g
Internal solid infill 11m4s 5.9% 2.44 m 7.44 g
Top surface 2m24s 1.3% 0.53 m 1.61 g
Bottom surface 2m33s 1.4% 0.48 m 1.46 g
Bridge 3m33s 1.9% 0.68 m 2.07 g
Skirt 28s 0.3% 0.07 m 0.23 g
Custom 13s 0.1% 0.02 m 0.05 g
Travel 5m45s 3.1%
Retract
Unretract
Wipe
Seams
Total Estimation
Total Filament: 39.59 m 120.94 g
Model Filament: 39.59 m 120.94 g
Cost: 3.63
Prepare time: 10s
Model printing time: 3h6m
Total time: 3h6m
Brim for better adhesion
Did you know that when printing models have a small contact interface with the printing surface, it's recommended to use a brim?
For more information, please check out our Wiki
Collapse
21/27

Wall Loops

- support for overhangs
- better finish



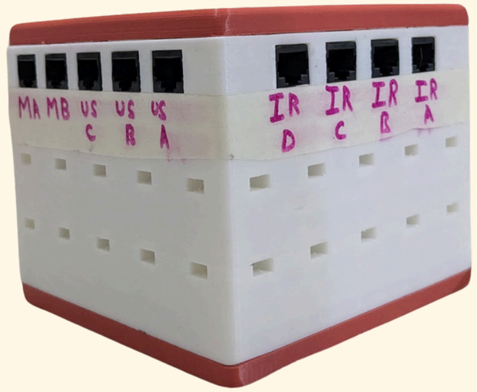


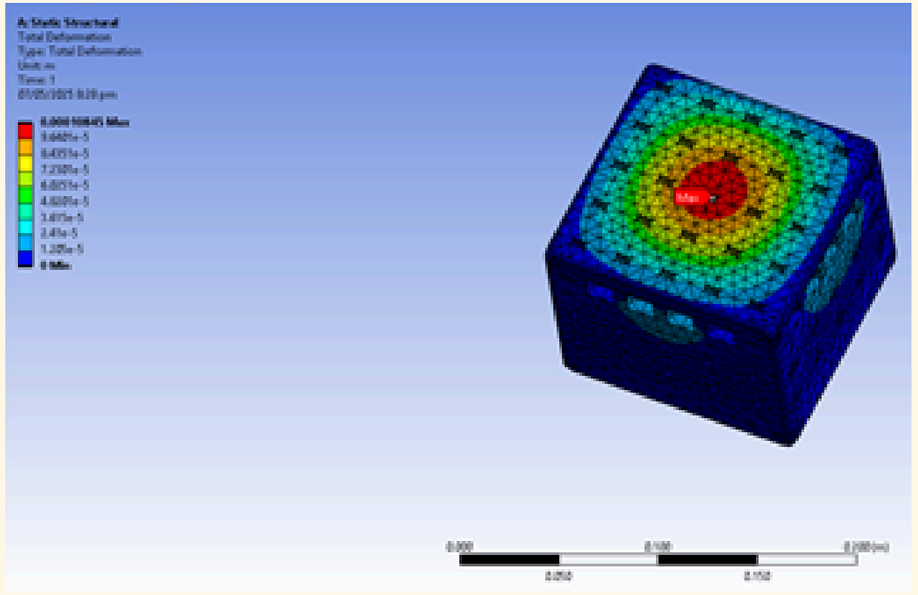
Power Brick

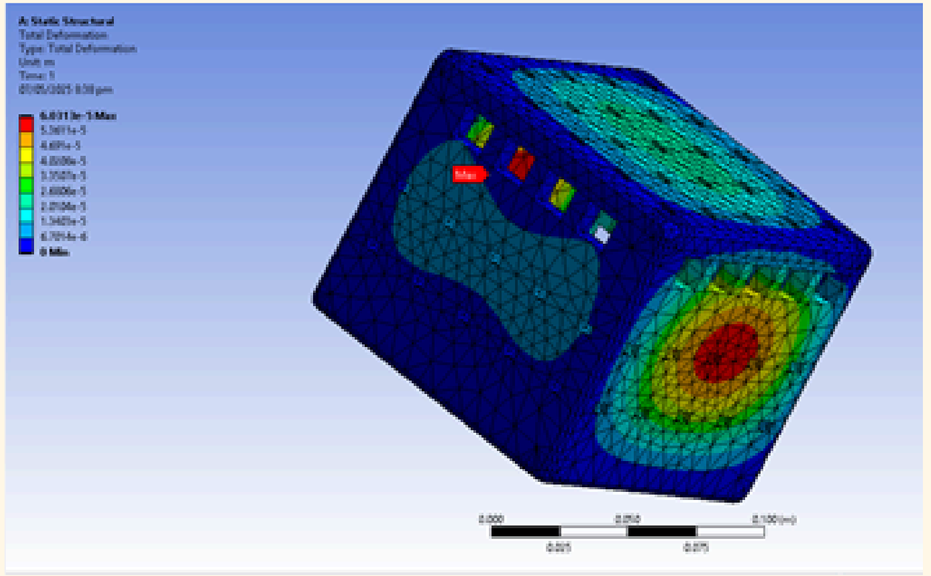
Features:

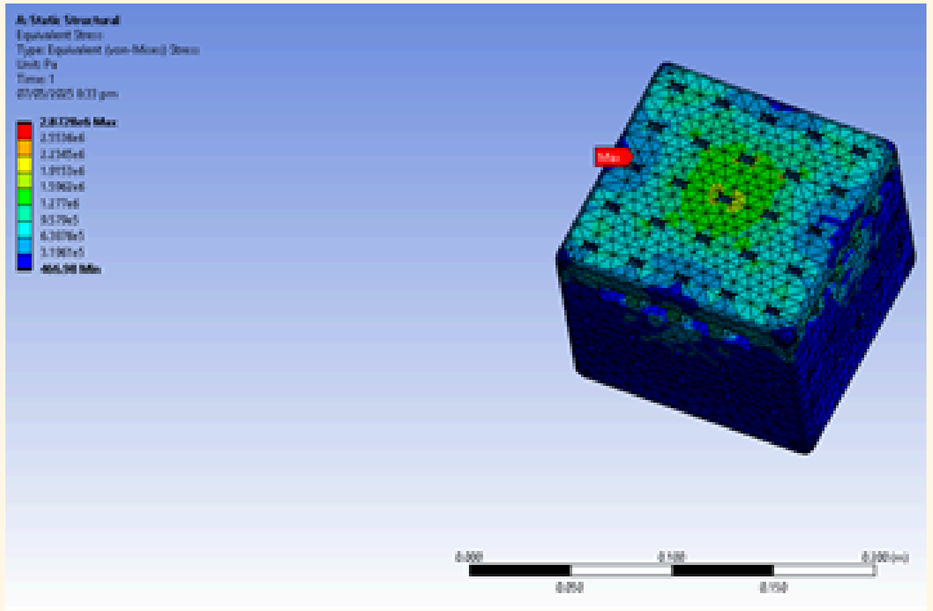
- 129.2mmx125mmx110mm
- 70 connecting slits
- Housing for Raspberry Pi 5, ESP 32-S3, PCB, Battery, Buck Convertors
- Access to Battery Charging Ports, Raspberry Pi 5 external ports, USB-to-type C port

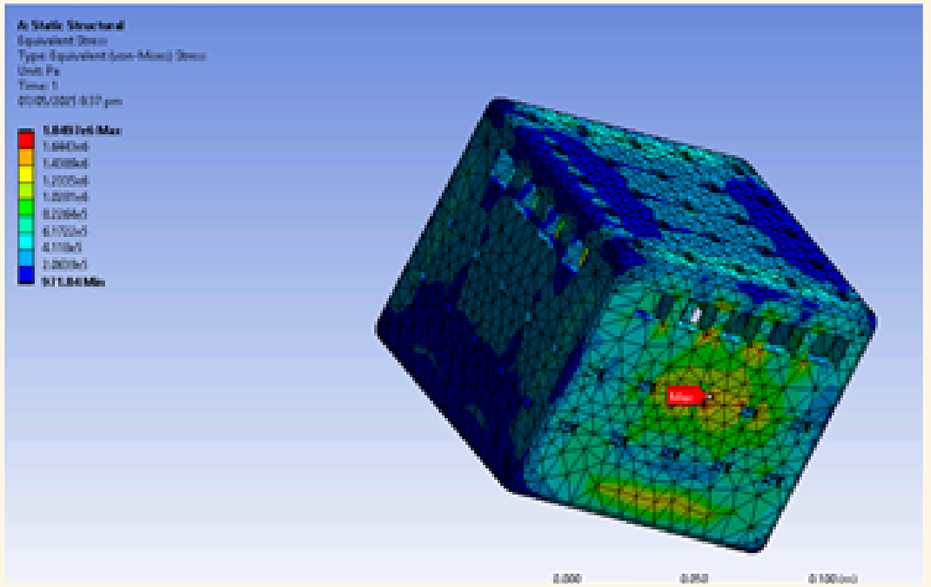





A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1
Max


A: Static Structural
Total Deformation
Type: Total Deformation
Unit: m
Time: 1
Max


A: Static Structural
Equivalent Stress
Type: Equivalent (von-Mises) Stress
Unit: Pa
Time: 1
Max


A: Static Structural
Equivalent Stress
Type: Equivalent (von-Mises) Stress
Unit: Pa
Time: 1
Max

Power Brick

$E = 3.5 \times 10^9\ Pa$ (Young's Modulus for PLA)

$\delta = 0.15mm$; (Deflection per arm (8.2-7.9)/2)

$I = \frac{\pi}{64} \times d^4 = 12.57 \times 10^{-12}\ m^4$;

$F = \frac{3EI\delta}{L^3}$;(Bending Beam Formula)

$F_y = 7.6N\ per\ arm$;

$\mu = 0.4$; (Coefficient of friction for PLA on PLA)

$F_x = \mu \times F_y$;

$F_x = 6.1N\ insertion\ force$;

Unsaved Project - Workbench

File Edit View Tools Units Extensions Jobs Help

Project | A2: Engineering Data

Filter Engineering Data | Engineering Data Sources

Toolbox

Field Variables
- Temperature
- Mean Stress
- R-Ratio

View All / Customize...

Outline of Schematic A2: Engineering Data

	A	B	C	D	E
1	Contents of Engineering Data			Source	Description
2	Material				
3	PLA				
4	Structural Steel			Gen	Fatigue Data at zero mean stress comes from 1998 ASME BPV Code, Section 8, Div 2, Table 5-110.1
*	Click here to add a new material				

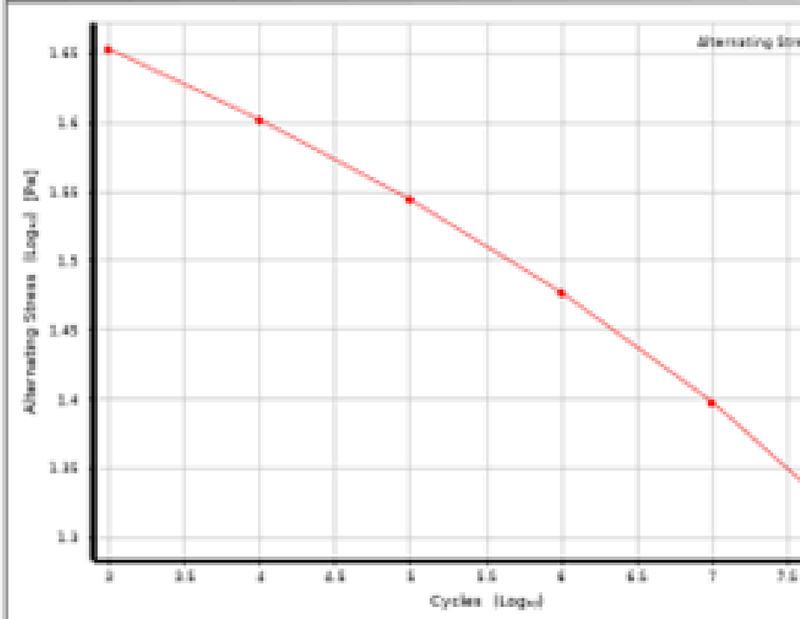
Table of Properties Row 10: S-N Curve

	A	B
1	Cycles	Alternating Stress (Pa)
2	1000	45
3	10000	40
4	1E+05	35
5	1E+06	30
6	1E+07	25
7	1E+08	20
*		

Properties of Outline Row 4: PLA

	A	B	C	D	E
1	Property	Value	Unit		
2	Material Field Variables	Table			
3	Density	1250	kg m^-3		
4	Isotropic Elasticity				
5	Derive from	Young's Modulus and Poisson...			
6	Young's Modulus	3.5	GPa		
7	Poisson's Ratio	0.36			
8	Bulk Modulus	4.1667E+09	Pa		
9	Shear Modulus	1.2868E+09	Pa		
10	S-N Curve	Tabular			
14	Tensile Yield Strength	50	MPa		
15	Tensile Ultimate Strength	65	Pa		
16	Isotropic Thermal Conductivity	0.13	W m^-1 K^-1		
17	Specific Heat Constant Pressure, C_p	1800	J kg^-1 K^-1		

Chart of Properties Row 10: S-N Curve



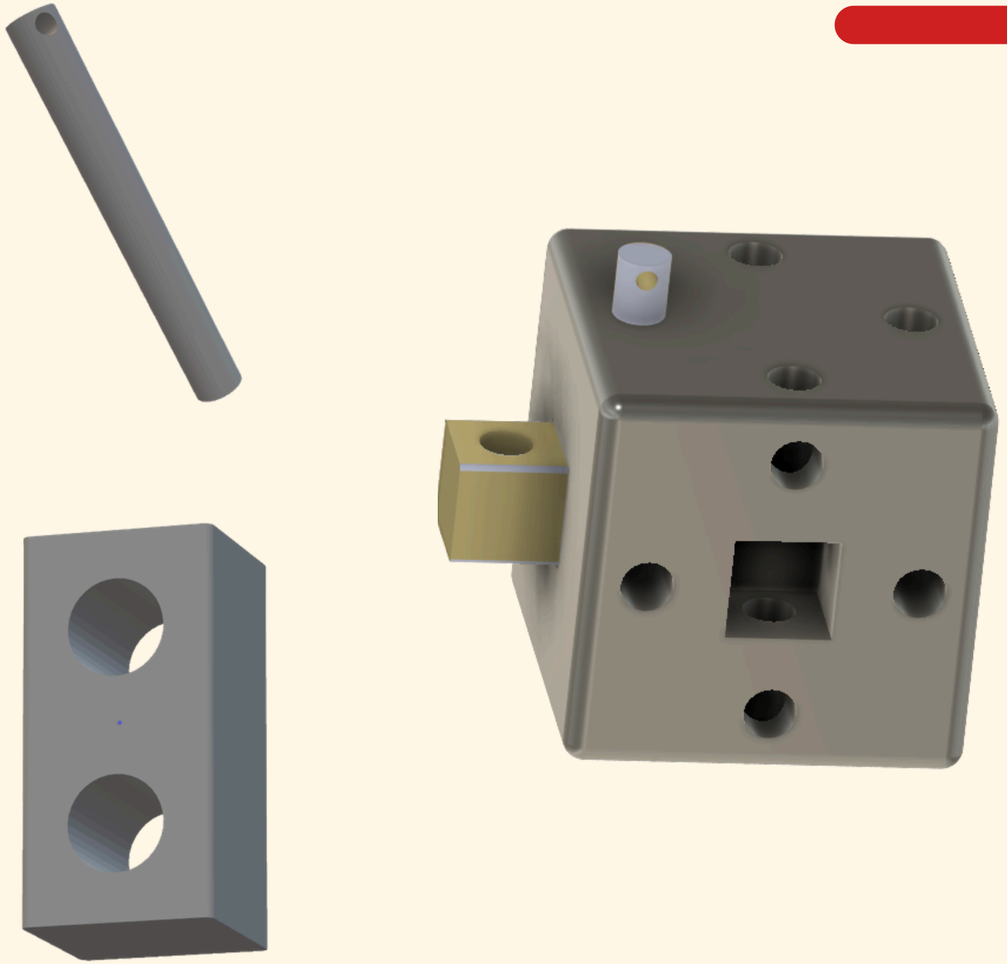
Ready

Job Monitor... | No DPS Connection | Show Progress | Show 0

Modular Building Blocks & Pin Recap

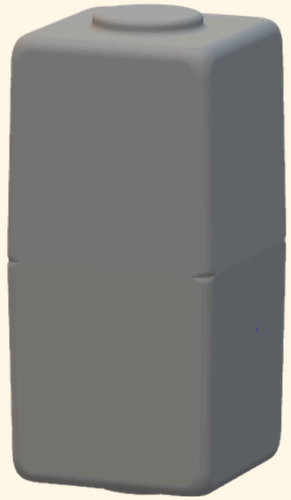
Recommendations based on Mid-Defense:

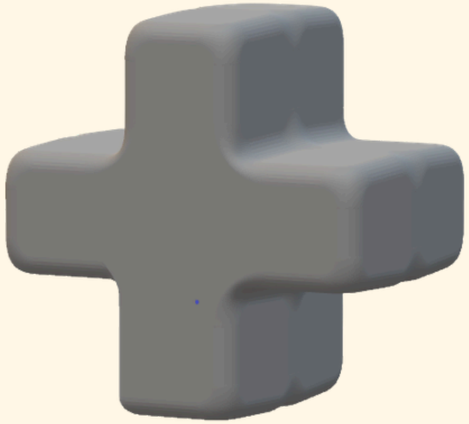
- Reduce the size of the block
- Reduce the connector to a one-step connection

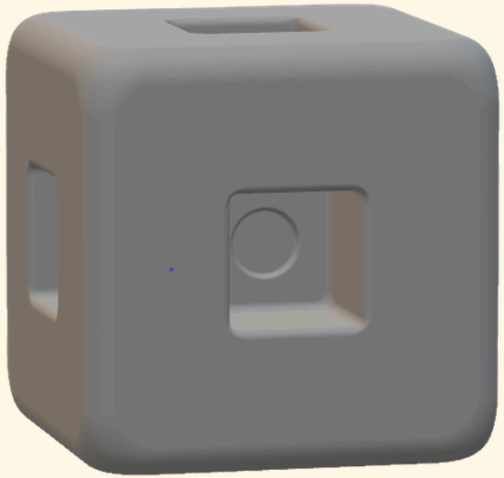


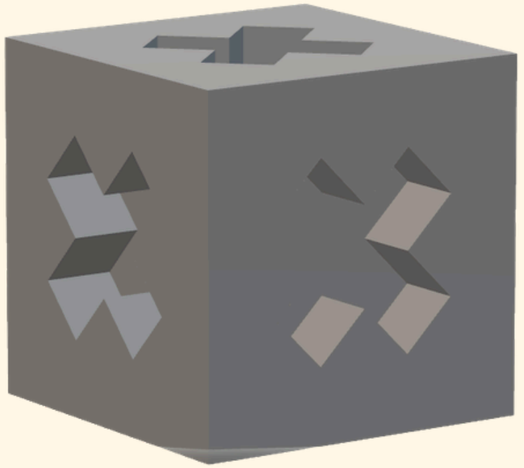
Evolution of Pin

Form and Press Fit Pins



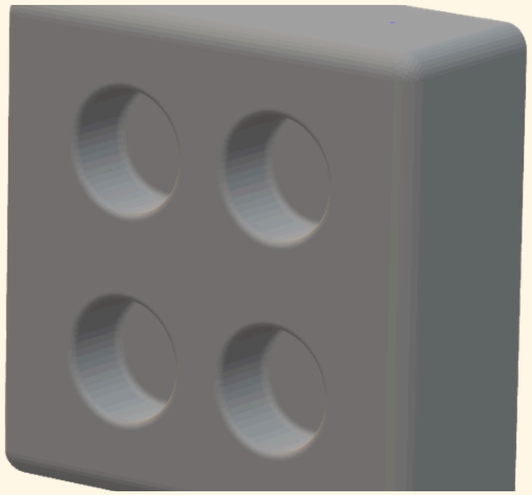


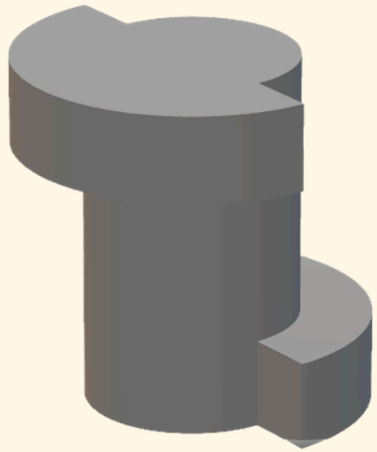


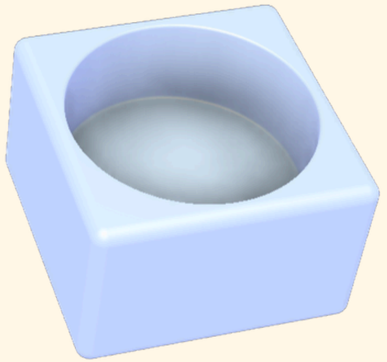


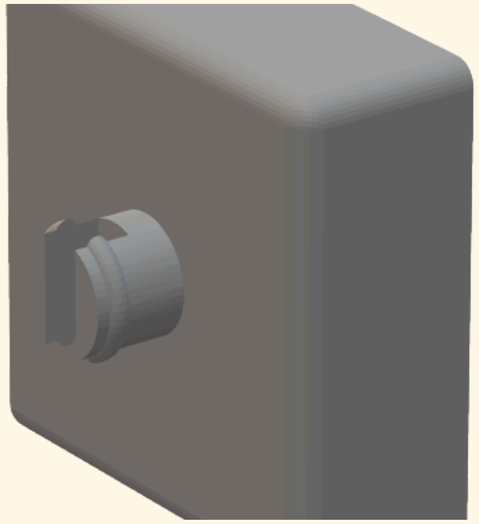
Evolution of Pin

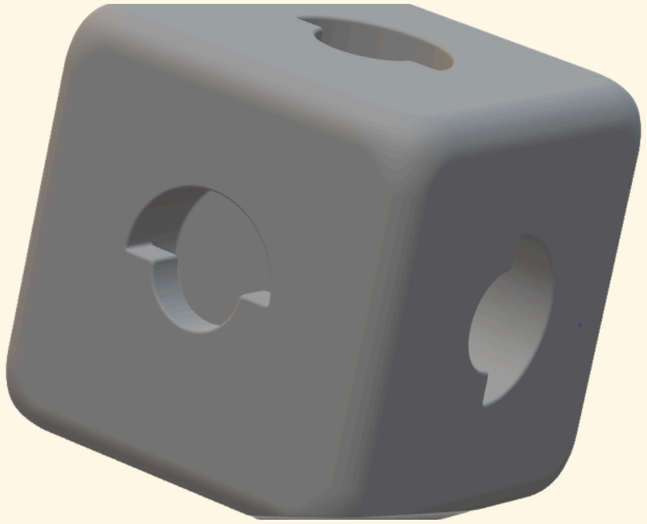
Press Fit, Rotational and Magnetic

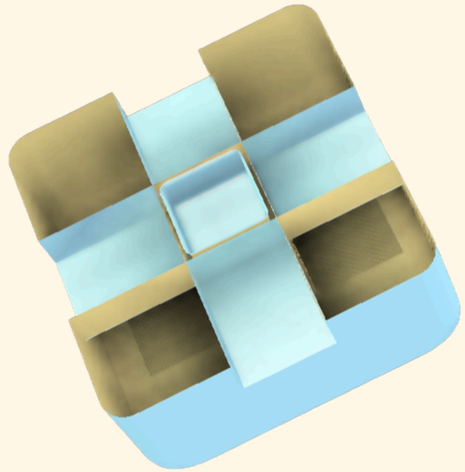






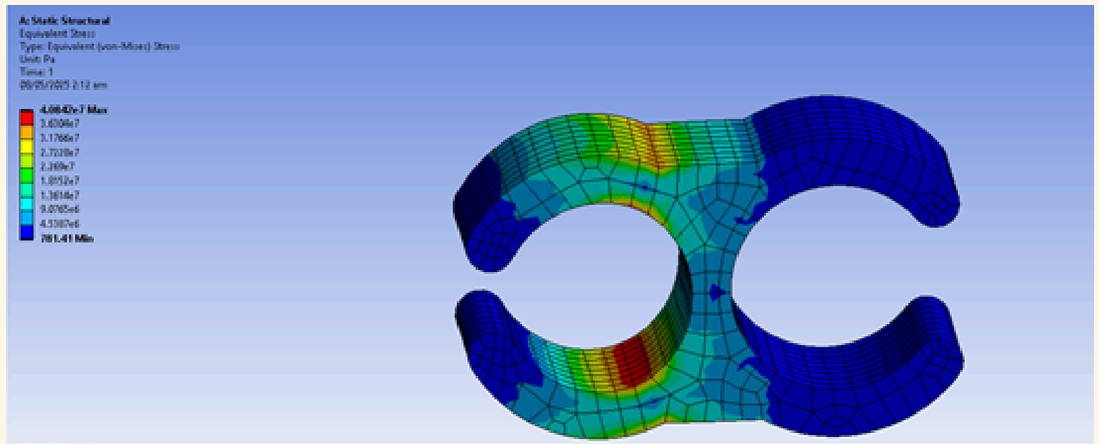


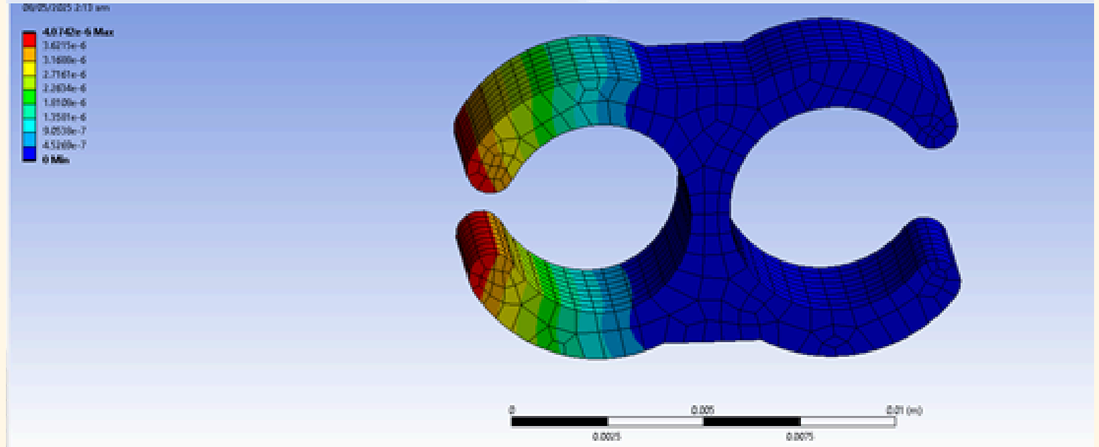


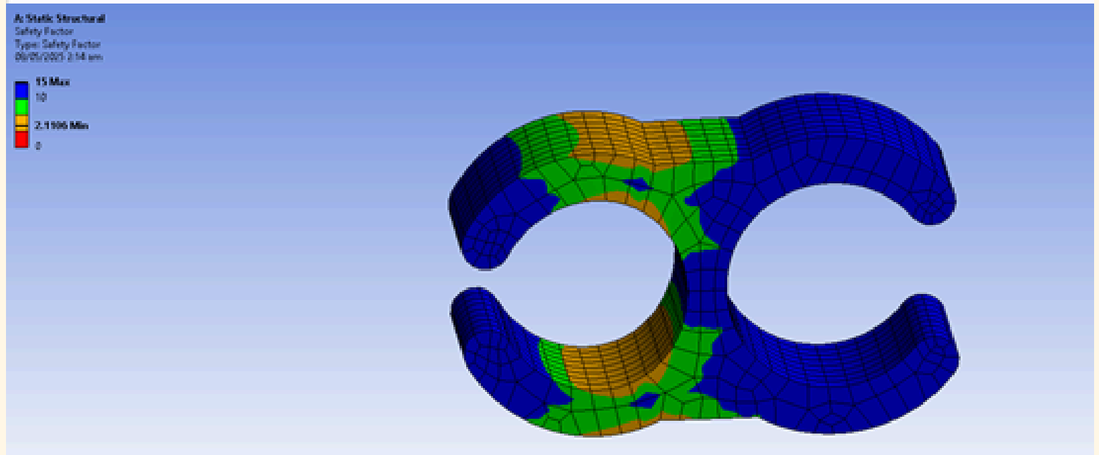


Final Pin

- ·The maximum stress is 40.7 MPa, which is significantly less than the ultimate yield strength.
- ·The maximum deflection of the beam is 0.4um.
- ·The factor of safety under these conditions are 2.11046.

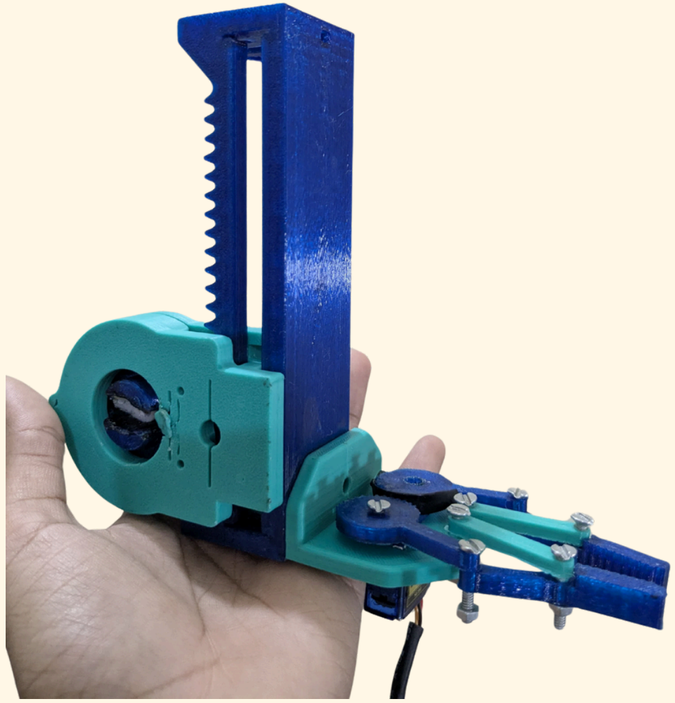




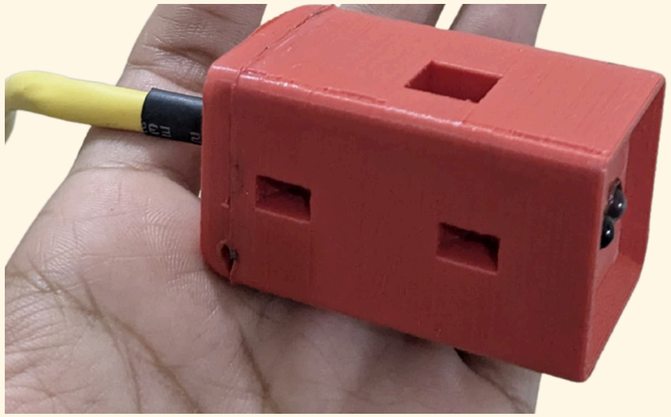


2 DOF Gripper





Sensor Blocks



IR Sensor Block

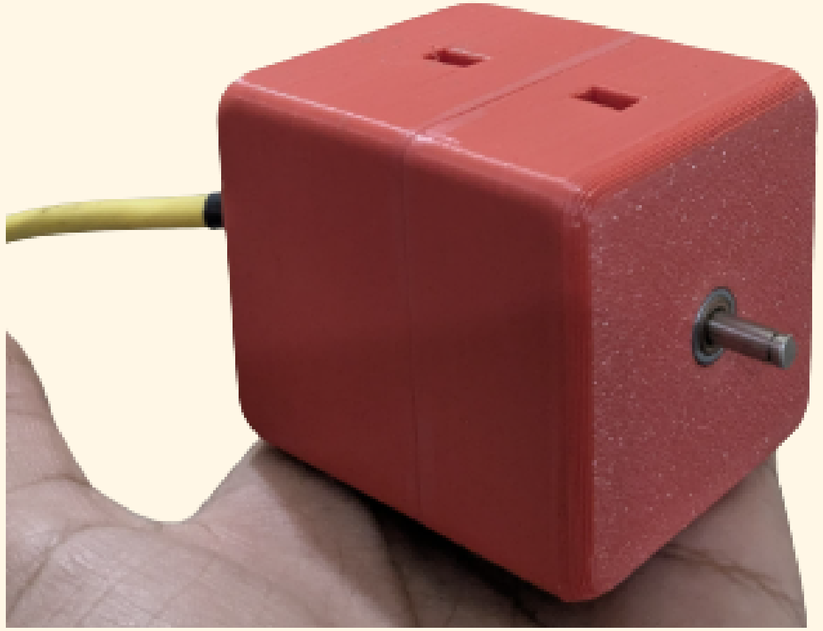


Ultrasonic Sensor Block

Sensor and Motor Blocks



Color Sensor Block

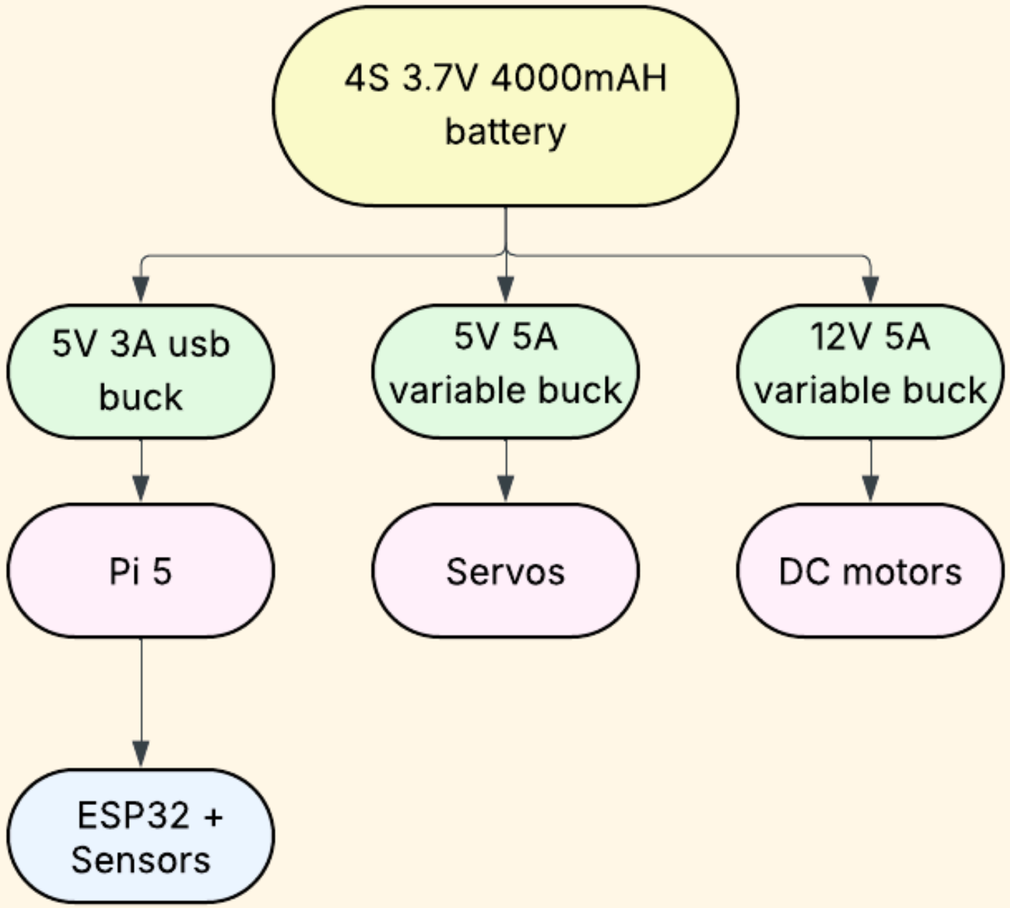


Motor Block

Power Architecture

Battery Configuration

- 4S Li-ion Pack (Each cell of 3.7v 4000mAh)
- Total Energy: 59.2Wh
- Chosen for high voltage headroom and moderate size



Power Consumption

- **Battery Energy**: (3.7V x 4) x 4Ah = 59.2Wh
- **Power Draw:** ~41W total (motors, Pi 5 w/ML, webcam, sensors)
- **Estimated Runtime:** 59.2/41 = 1.44H (86 Minutues)

Index	Component	Voltage (V)	Current (A)	Power (W)
0	HW-201 Digital IR Sensors (4x)	3.3	0.08	0.264
1	HC-SR04 Ultrasonic Sensors (3x)	3.3	0.045	0.1485
2	TCS3200 Color Sensor	3.3	0.02	0.066
3	ESP32-S3	3.3	0.06	0.198
4	SG90 Servos (2x)	5	0.4	2
5	Raspberry Pi 5	5	2.8	14
6	12V DC Motors (2x) 25GA370	12	2	24
Total	Total Power Draw	35.2	5.405	40.6765

RJ45: Wiring Connector

Reason for Selection

- 8 Pins Available — enough for most sensors and actuators, including the TCS3200 color sensor (needs 7 pins: 4 control + 1 output + VCC + GND)
- Child-Friendly
- Cost-Effective
- JST, Dupont Wires: small and cheap, but fragile and not ideal for repeated use



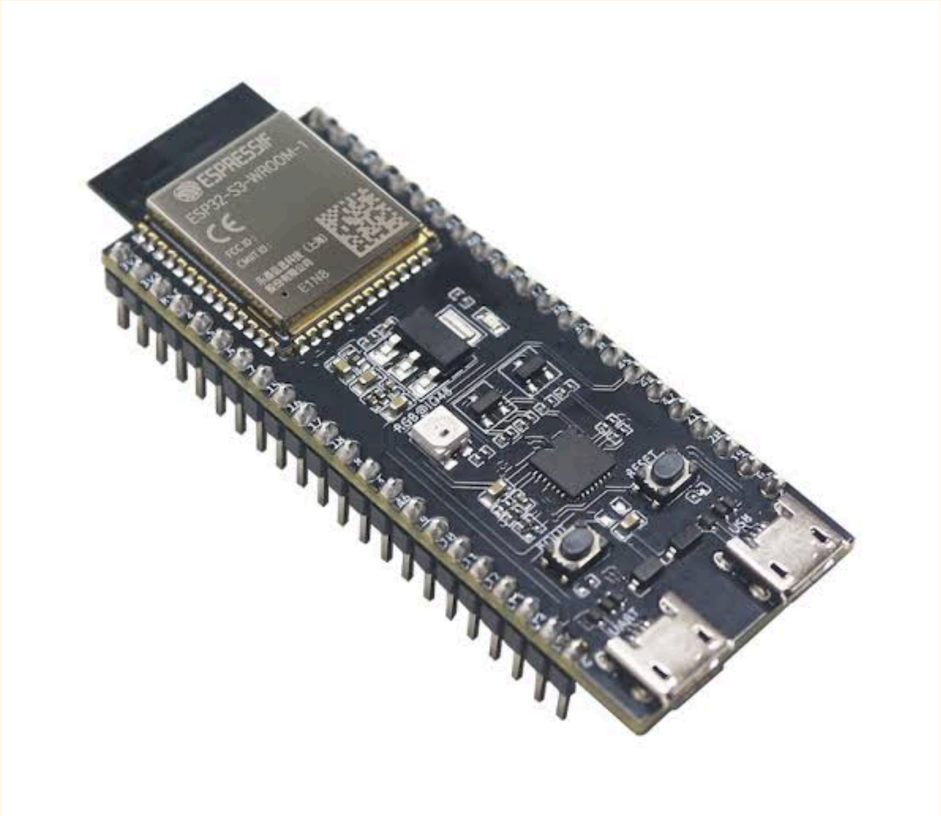
Controller: Reasons ESP32-S3 is managing the low level functions

Why?

- ROS2 introduced latency in sensor reading due to it being run on a OS.

Solution

- Switched to ESP32-S3 microcontroller with 40 GPIO pins.
- Ran code as embedded firmware to reduce latency drastically.



Build-It Library: Purpose and Use

- Simplify hardware control for students using Blockly-generated code.

How it works:

- Students only need to call setup_hardware() in setup() function.
- All sensor and actuator functions are exposed through one library.
- Abstracts complexity behind simple function calls like run_motor_at_speed(), read_ir_light().

```
1   #include <BuildIT.h>
2
3   void setup() {
4       setup_hardware();
5   }
```

FreeRTOS in Build-It Library

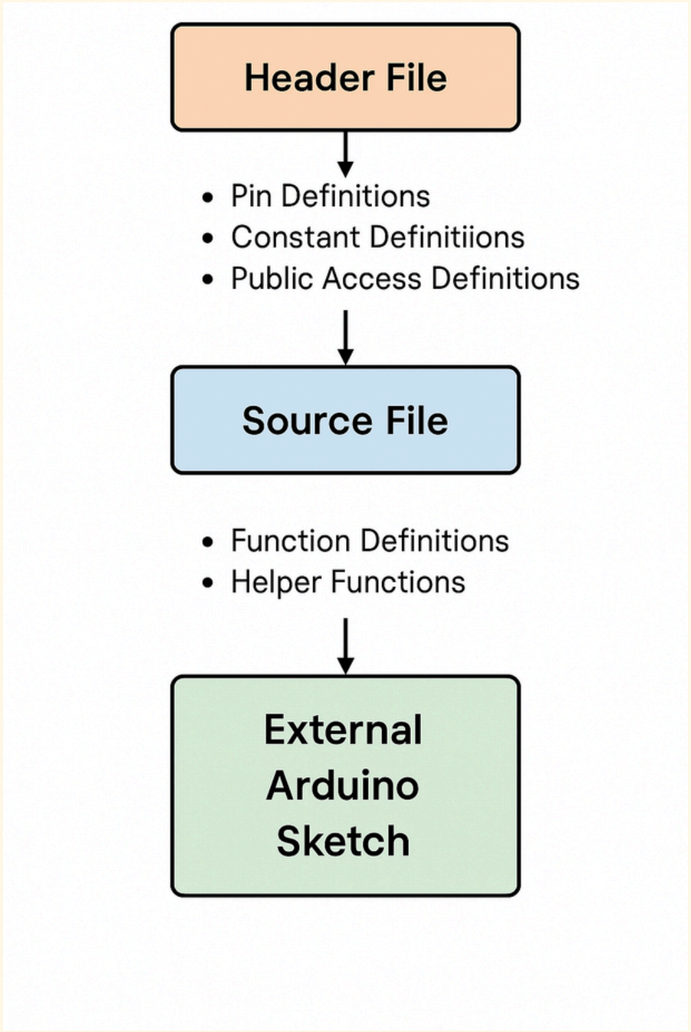
- **Why was it used?**
- IR and Ultrasonic sensors require constant polling for smooth results.
- FreeRTOS tasks us_task() and fast_ir_task() continuously read sensors in background.
- Maintains rolling buffer of 5 readings to return a stable average on request.

- **Why not used elsewhere?**
- Motors, servos, camera triggers are event-driven and don't need constant polling.
- Simpler and more efficient to run them directly when needed.

```
void fast_ir_task(void *param) {
  while (true) {
    for (int i = 0; i < 4; i++) {
      portENTER_CRITICAL(&ir_mux);
      ir_values[i][ir_idx[i]] = digitalRead(IR_PINS[i]);
      ir_idx[i] = (ir_idx[i] + 1) % MAX_READINGS;
      portEXIT_CRITICAL(&ir_mux);
    }
    vTaskDelay(5 / portTICK_PERIOD_MS); // Fast update rate for IR sensors
  }
}
```

Library Structure

- The student calls a function in the main arduino sketch which was generated from the app
- The sketch looks up to the header file for the public access function declaration
- During compilation this function is linked to the source file where it is defined



Benefits of Build-It Library

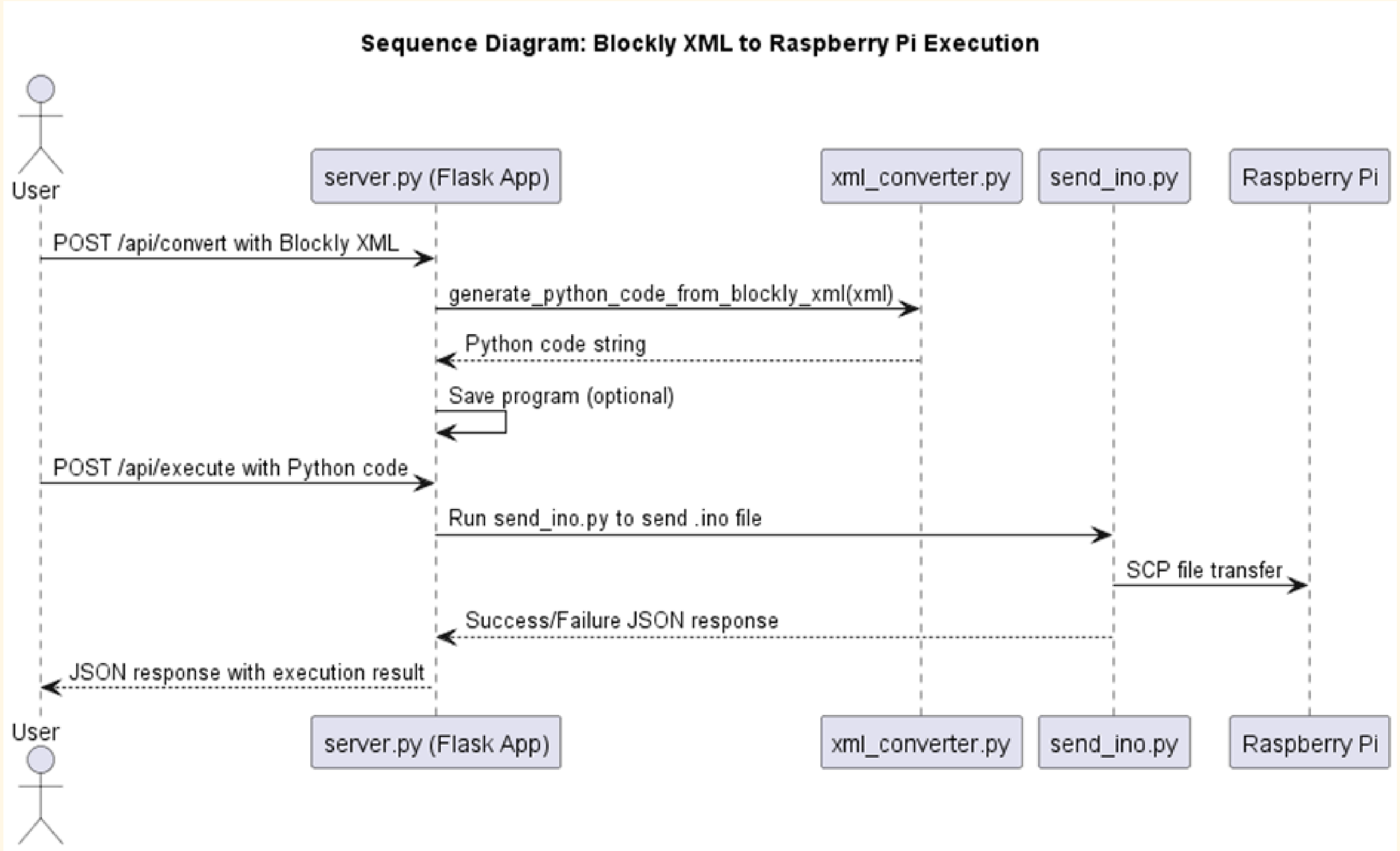
User Benefits

- No need to write functions for sensing and actuating tasks.
- Use of Blockly simplifies programming logic.
- Real-time averaged sensor data improves accuracy and responsiveness.

Technical Achievements

- Clean abstraction using C++ and FreeRTOS.
- Modular and expandable design.
- Smooth integration with blockly generated code

USER APPLICATION


Sequence Diagram: Blockly XML to Raspberry Pi Execution
User
server.py (Flask App)
xml_converter.py
send_ino.py
Raspberry Pi
POST /api/convert with Blockly XML
generate_python_code_from_blockly_xml(xml)
Python code string
Save program (optional)
POST /api/execute with Python code
Run send_ino.py to send .ino file
SCP file transfer
Success/Failure JSON response
JSON response with execution result
User
server.py (Flask App)
xml_converter.py
send_ino.py
Raspberry Pi

Flask-Based Backend

- Used a lightweight, flexible web framework for building the web app
- Managing client requests, routing, and server-side logic.
- / and /editor - Serve static HTML files for the main interface.
- /api/convert - Accepts Blockly XML, converts it to C++ code(using xml_converter)
- Flask triggers Python scripts (send_ino.py) using subprocess.run() to execute user-generated code.

```python
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=8000)
```

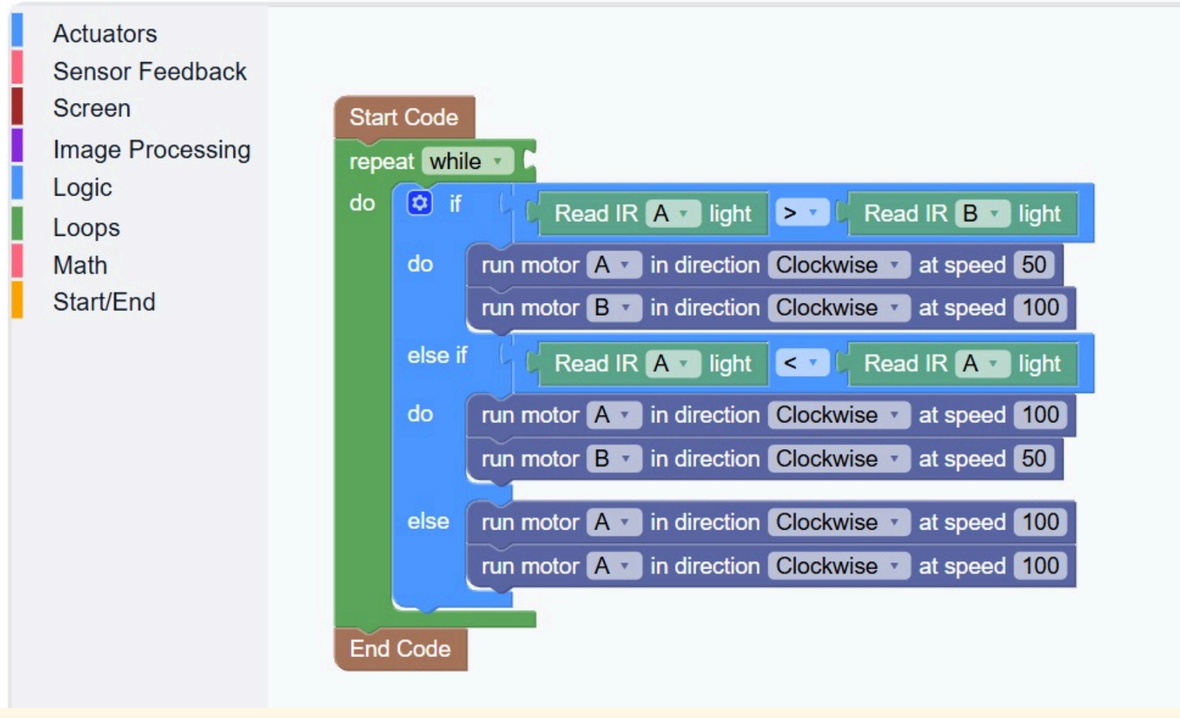
```python
@app.route('/editor')
def editor():
    try:
        return send_file('editor.html')
    except Exception as e:
        logger.error(f"Error serving editor page: {str(e
        return jsonify({'error': str(e)}), 500
```

```python
@app.route('/api/save', methods=['POST'])
def save_program():
    """Save a program, updating it if the name already exists."""
    try:
        data = request.json
        if not data or not all(k in data for k in ['name', 'xml',
            return jsonify({'error': 'Missing required fields'}),

        # Check for existing program with the same name
        existing_program = None
        for filename in os.listdir(PROGRAMS_DIR):
            if filename.endswith('.json'):
                file_path = os.path.join(PROGRAMS_DIR, filename)
                with open(file_path, 'r') as f:
                    program_data = json.load(f)
                    if program_data.get("name") == data["name"]:
                        existing_program = file_path
                        break  # Stop searching once found
```

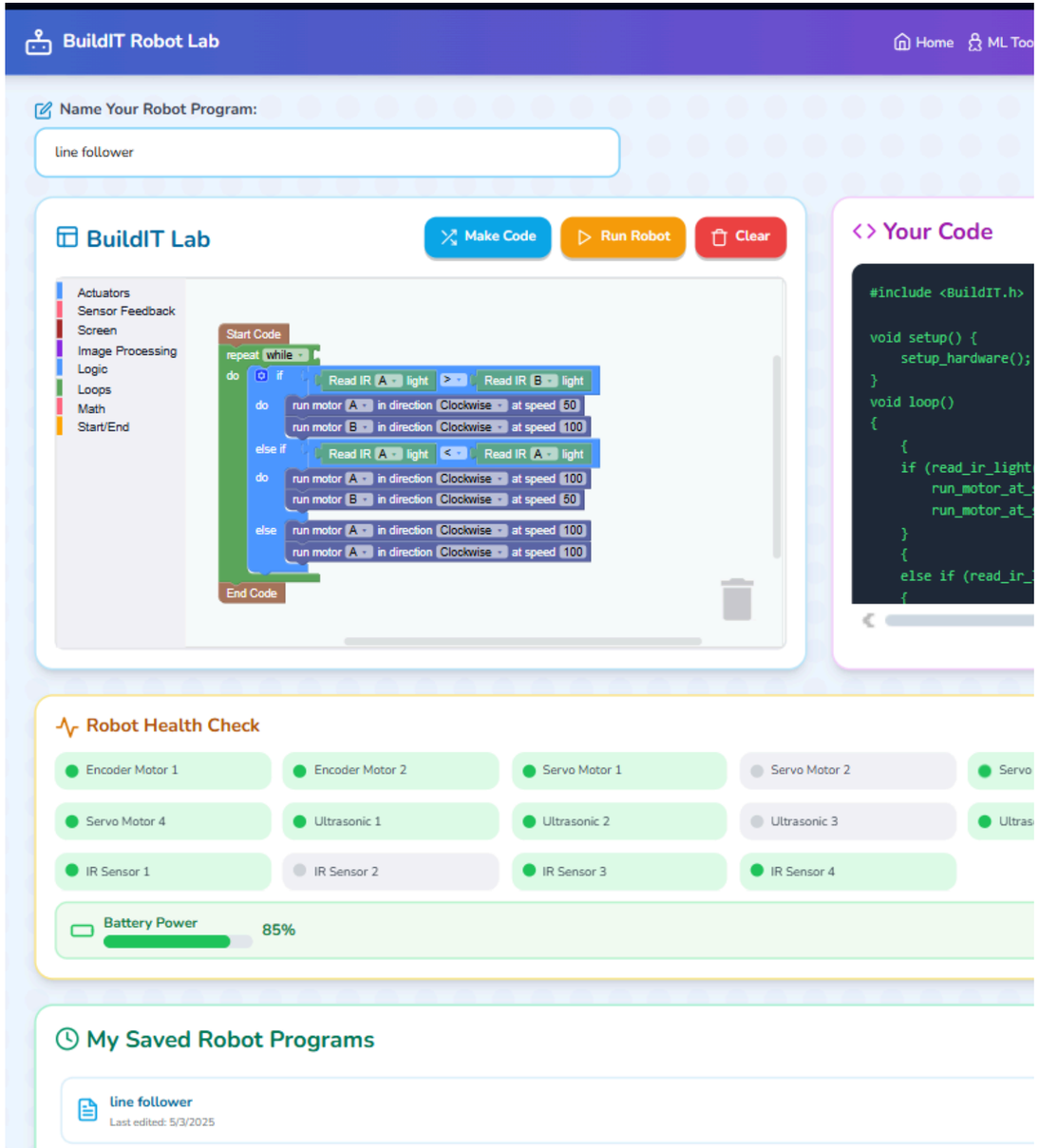
User Application

- Integrated Blockly’s core library via CDN links.
- Defined a custom toolboxes (actuators,sensors)that contain my robot-specific blocks
- Created my own block types in Blockly using JavaScript.
- run_motor, move, stop_motor, display_text, controls_if, etc.




```
Blockly.Blocks['stop_movement'] = {
    init: function() {
        this.appendDummyInput()
            .appendField("Stop movement");
        this.setPreviousStatement(true, null);
        this.setNextStatement(true, null);
        this.setColour(120);
        this.setTooltip("");
        this.setHelpUrl("");
    }
};

Blockly.Blocks['read_ir_light'] = {
    init: function() {
        this.appendDummyInput()
            .appendField("Read IR")
            .appendField(new Blockly.FieldDropdown([["A","A"], ["B","B"], ["C","C
            .appendField("light");
        this.setOutput(true, "Number");
        this.setColour(160);
```


BuildIT Robot Lab
Home
ML Too
Name Your Robot Program:
line follower
BuildIT Lab
Make Code
Run Robot
Clear
Actuators
Sensor Feedback
Screen
Image Processing
Logic
Loops
Math
Start/End
Start Code
repeat while
do
if
Read IR A light > Read IR B light
do
run motor A in direction Clockwise at speed 50
run motor B in direction Clockwise at speed 100
else if
Read IR A light < Read IR A light
do
run motor A in direction Clockwise at speed 100
run motor B in direction Clockwise at speed 50
else
run motor A in direction Clockwise at speed 100
run motor A in direction Clockwise at speed 100
End Code
Your Code
#include <BuildIT.h>
void setup() {
setup_hardware();
}
void loop()
{
{
if (read_ir_light
run_motor_at_
run_motor_at_
}
{
else if (read_ir_
{
Robot Health Check
Encoder Motor 1
Encoder Motor 2
Servo Motor 1
Servo Motor 2
Servo
Servo Motor 4
Ultrasonic 1
Ultrasonic 2
Ultrasonic 3
Ultras
IR Sensor 1
IR Sensor 2
IR Sensor 3
IR Sensor 4
Battery Power
85%
My Saved Robot Programs
line follower
Last edited: 5/3/2025

JavaScript

- Used Blockly.Xml.workspaceToDom() and Blockly.Xml.domToText() to convert the current Blockly workspace into an XML string.
- Used document.getElementById() to target specific buttons (saveBtn, convertBtn, executeBtn).
- Applied addEventListener() to these buttons: Save, Convert, Execute
- Implemented a showNotification() function for dynamic, user-friendly alerts (success, warning, error).

```
nst xml = Blockly.Xml.workspaceToDom(workspace
nst xmlText = Blockly.Xml.domToText(xml);
    //Execute program
    document.getElementById('executeBtn').addEventListener('click', async () => {
const code = document.getElementById('codeOutput').textContent.trim();
if (!code) {
    showNotification('First convert your blocks to code!', 'warning');
    return;
}

try {
    showNotification('Running your robot program...', 'info');

    const response = await fetch('/api/execute', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
        },
        body: JSON.stringify({ code }),
    });

    const result = await response.json();

    if (result.success) {
        // Success case
        showNotification(result.message || 'Your robot program is running!', 'success

// Show notification
function showNotification(message, type = 'info') {
    // Create notification element
    const notification = document.createElement('div');
    notification.className = 'fixed bottom-4 right-4 px-6 py-4 rounded-xl shadow-lg flex items-
```

XML to C++ Converter

- Utilized xml.etree.ElementTree (ET) in Python to parse Blockly XML.
- Translated each Blockly block (e.g., run_motor, move) into corresponding C++ code using a parse_block() function.
- Supported logic blocks (controls_if, controls_repeat_ext) and sensor blocks (read_ultrasonic_distance).



```
try:
    # Parse the XML
    root = ET.fromstring(xml_string)
    python_code = []  # Store code lines in a list for better management
    print("XML Root:", ET.tostring(root, encoding="unicode"))

    def parse_block(block):
        if block is None:
            return "100"

        namespace = {'ns0': 'https://developers.google.com/blockly/xml'}  # Namespace
        block_type = block.attrib.get("type")
        print(f"Processing Block: {block_type}")  # Debugging output

        # FIX: Extract fields, values, and statements correctly
        fields = {field.attrib["name"]: field.text for field in block.findall("ns0:fi
        values = {}


        elif block_type == "read_ultrasonic_distance":
            sensor = fields.get("SENSOR", "A")
            return f"read_ultrasonic_distance({sensor})"


    elif block_type == "controls_if":
        i=0
        while f"IF{i}" in values:
            condition = values.get(f"IF{i}", '('+"True"+')')
            body = statements.get(f"DO{i}")
            code_lines.extend('{')
            if i == 0:
                code_lines.append(f"if {condition}"+'{')

            else:
                code_lines.append(f"else if {condition.rstrip(';')}")
                code_lines.extend('{')
            if body:
                if isinstance(body, str):
                    body = body.splitlines()
                code_lines.extend([f"    {line}" for line in body])
                i += 1
            code_lines.extend('}')
```

Execute

- Defined the .ino file name and local folder path, then combined them to get the full file path.
- Checked if the file exists — if not, printed a JSON error message and exited.
- Set up remote Raspberry Pi details: SSH username, host, and destination folder path.
- Built the scp command to transfer the file over the network.
- Executed the command using subprocess.run(), handling success and error responses with JSON-formatted messages.

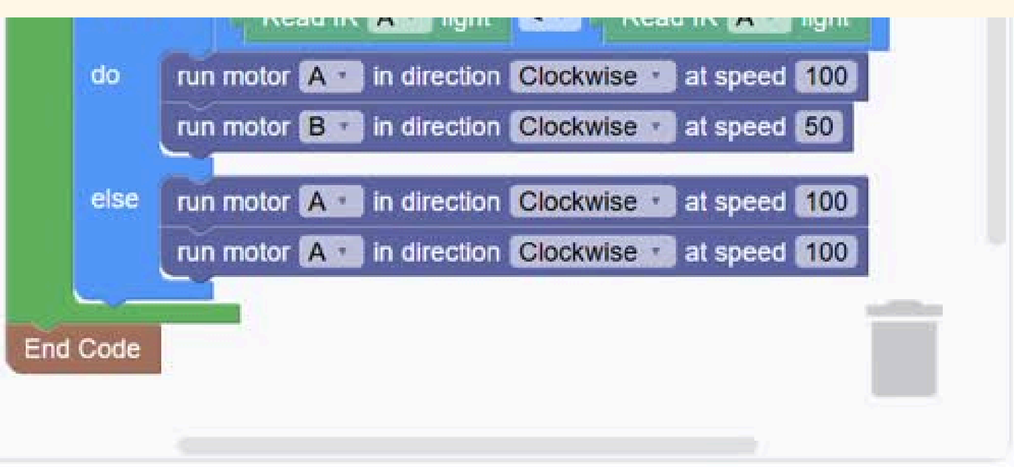
```
# === SETTINGS ===
file_name = "helo.ino"  # ✅ Ensure this file exists
local_folder = r"C:\Users\Ibrahim Ghazi\Desktop\FYDP\program\send_ino_file"
local_path = os.path.join(local_folder, file_name)

# === CHECK FILE EXISTS ===
if not os.path.exists(local_path):
    # Return JSON response for file not found
    print(json.dumps({
        "success": False,
        "message": f"❌ File not found at: {local_path}"
    }))
    exit(1)

# === REMOTE SETTINGS ===
remote_user = "buildit"
remote_host = "buildit.local"
remote_folder = "/home/buildit/builditapp/received_program"
remote_path = f"{remote_user}@{remote_host}:{remote_folder}"

# === RUN SCP COMMAND ===
scp_command = f'scp "{local_path}" {remote_path}'

try:
    subprocess.run(scp_command, check=True, shell=True)
    # Return JSON response for success
    print(json.dumps({
        "success": True,
        "message": f"✅ File '{file_name}' sent successfully to Raspberry Pi a
    }))
except subprocess.CalledProcessError as e:
    # Return JSON response for error
    print(json.dumps({
        "success": False,
        "message": f"❌ Failed to send file. Error: {str(e)}"
    }))
```




```
{
else if (read_ir_light('A')
{
    move('Forward', 100);
}
else
{
    move('Forward', 100);
```

Robot Health Check

Encoder Motor 1	Encoder Motor 2	Servo Motor 1	Servo Motor 2	Servo Motor 3
Servo Motor 4	Ultrasonic 1	Ultrasonic 2	Ultrasonic 3	Ultrasonic 4
IR Sensor 1	IR Sensor 2	IR Sensor 3	IR Sensor 4	

Battery Power 85%

My Saved Robot Programs

Loaded program: line follower

Choosing a Middleware Framework

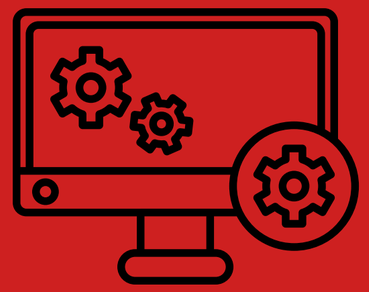


vs

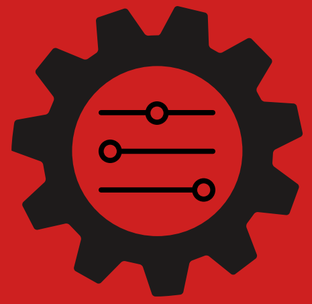
BuildiTOS

Why BuildiTOS (technically not an OS)

RESOURCE CONSTRAINTS



SIMPLER COMMUNICATION



FASTER BOOT TIME



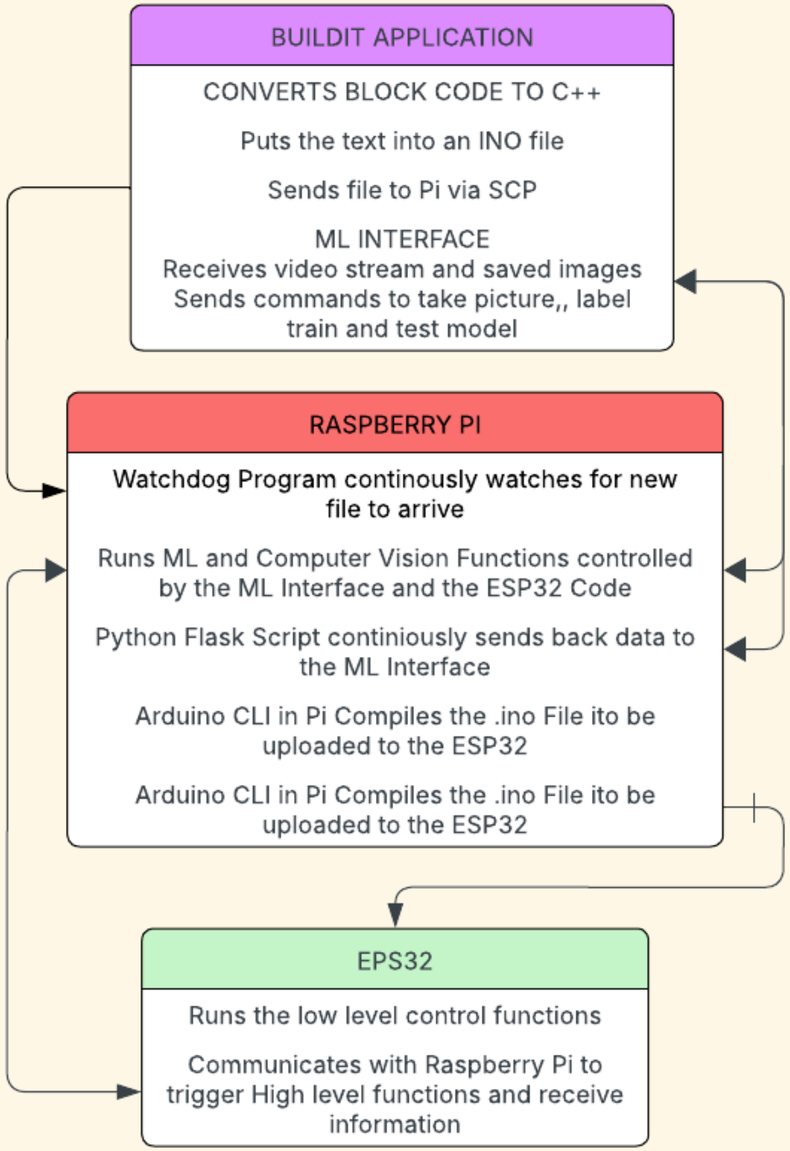
Dependency Management



REAL TIME RESPONSE



BuildiTOS Framework Overview



Local Network

- Hosted on a hotspot for internal communication.

App Code Pipeline

- Receives .ino code via Flask SCP from WebApp.
- Compiles .ino for ESP32.
- Uploads compiled code to ESP32.

ML Pipeline

- Receives video stream via MPEG on the local network.
- Accesses saved training images.
- Sends command to take picture (handled by Pi).
- Sends label for next 5 training images.
- Sends command to train on these images and label.
- Can reset current model training.

Robot Runtime

- ESP32 → Pi: GPIO signal to take picture for training.
- ESP32 → Pi: GPIO signal to take picture and run inference.
- Pi → ESP32: GPIO signal indicating detection result.

ESP32

- Manages low-level sensors and actuators.
- GPIO pins mapped to RJ45 connectors for interfacing.

MACHINE LEARNING

Few Shot Object Detection

Collecting and processing large data sets is tedious
Beginners Programmer can focus more on coming up with creative algorithms and solutions
Any More than 5 Pictures becomes difficult to manage for beginners

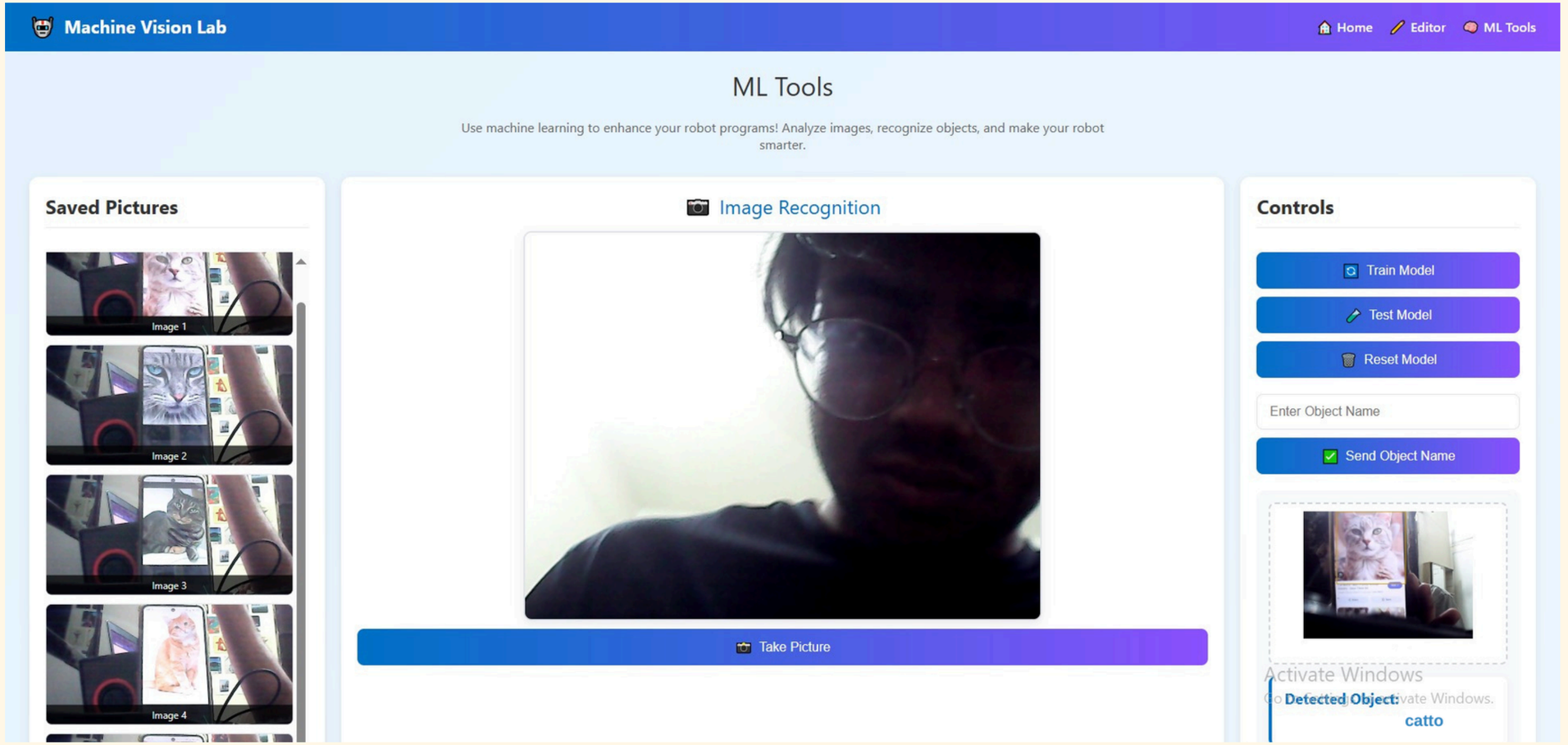
---

Take 5 images

Train

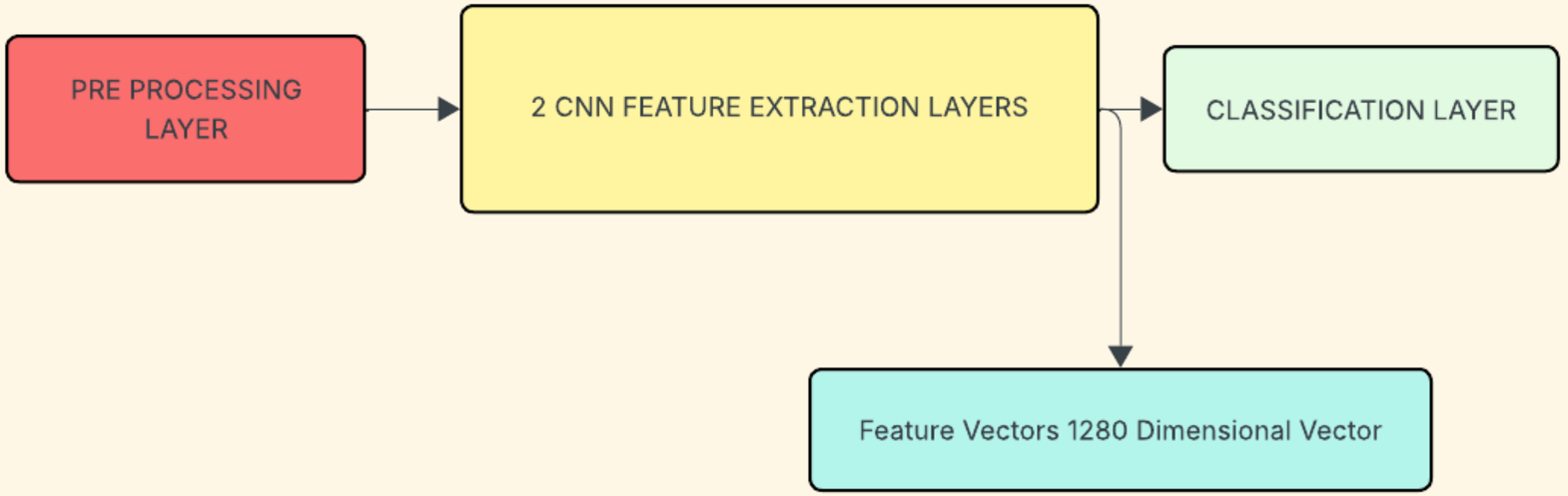
Test

ML Lab Interface

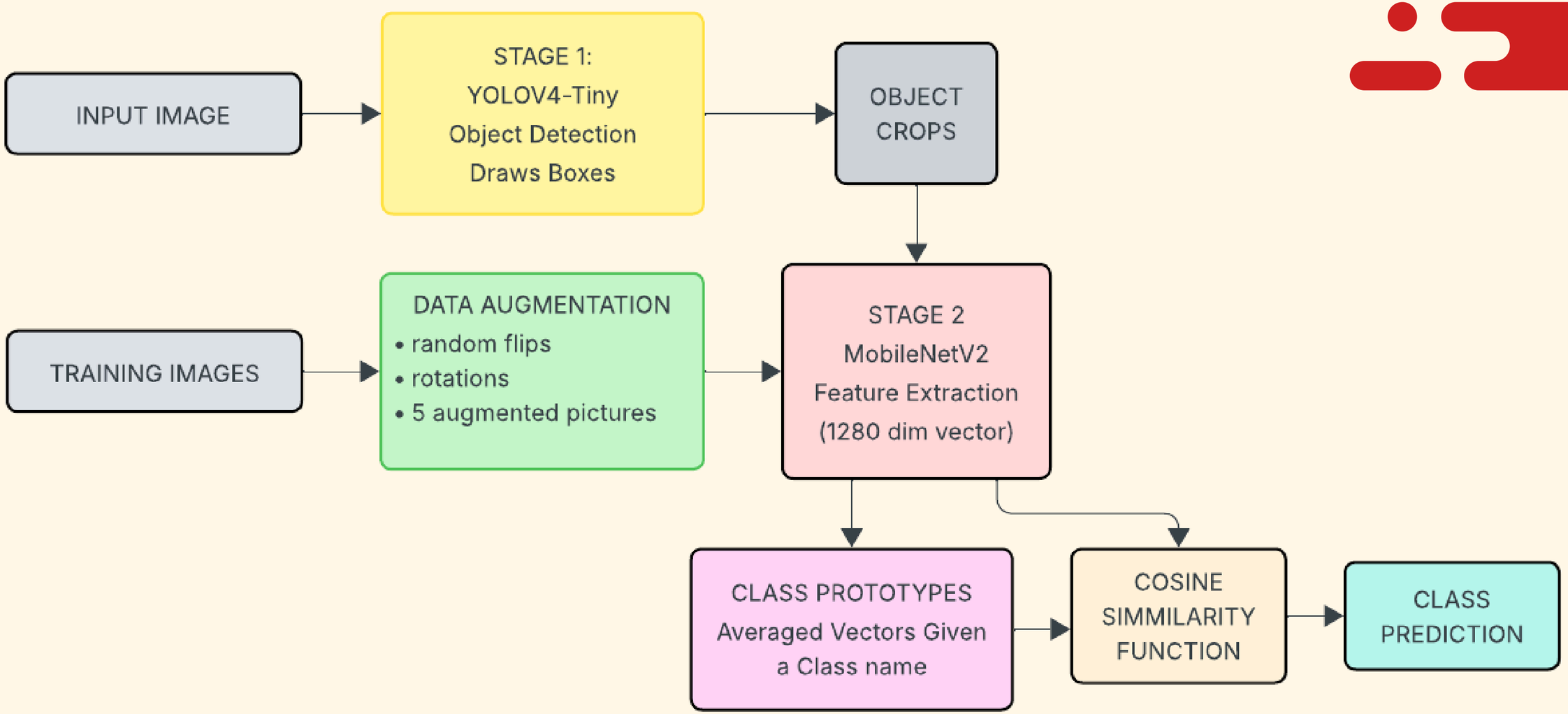


How it works

MobileNetV2 Architecture



Model Architecture



TESTING EXAMPLE

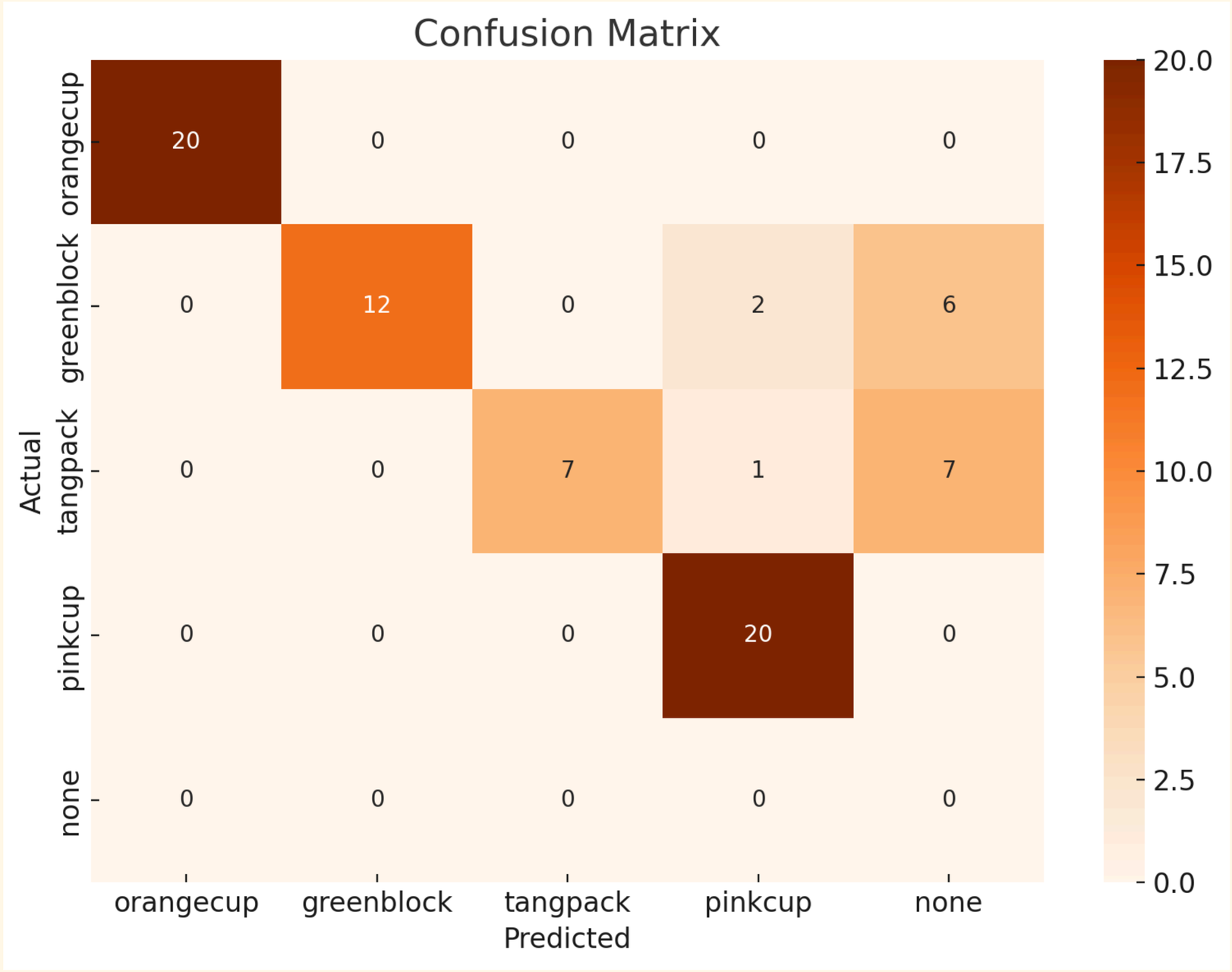








Class	TP	FP	FN
orangecup	20	0	0
greenblock	12	0	8
tangpack	7	0	8
pinkcup	20	3	0


Confusion Matrix
Actual
orangecup
greenblock
tangpack
pinkcup
none
20 0 0 0 0
0 12 0 2 6
0 0 7 1 7
0 0 0 20 0
0 0 0 0 0
orangecup
greenblock
tangpack
pinkcup
none
Predicted
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{59}{59 + 3} = \frac{59}{62} \approx 0.9516$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{59}{59 + 21} = \frac{59}{80} = 0.7375$$

$$\text{F1 Score} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

$$= \frac{2 \cdot (0.9516 \cdot 0.7375)}{0.9516 + 0.7375} = \frac{2 \cdot 0.7019}{1.6891} \approx 0.8304$$

Class	TP	FP	FN	Precision	Recall	F1 Score
Orangecup	20	0	0	1	1	1
Greenblock	12	0	8	1	0.6	0.75
Tangpack	7	0	8	1	0.467	0.637
Pinkcup	20	3	0	0.87	1	0.93

Thankyou